

Contents

M3 - Parte práctica	5
Recorridos de entorno	5
Consola M3	5
IntelliJ IDEA M3	5
Eclipse M3	5
VS Code M3	6
Práctica 3.1 - GET: consultar recursos	6
Paso 1 - Repasar el estado actual	6
Adaptación al snapshot final M3	6
Verificación	6
Paso 2 - Ampliar el filtrado en el servicio	6
Adaptación al snapshot final M3	7
Verificación	7
Paso 3 - Añadir ordenación al servicio	7
Adaptación al snapshot final M3	8
Verificación	8
Paso 4 - Añadir paginación al servicio	8
Adaptación al snapshot final M3	8
Verificación	8
Paso 5 - Añadir un método para contar elementos	8
Adaptación al snapshot final M3	9
Verificación	9
Paso 6 - Refactorizar el controlador para usar los nuevos parámetros	9
Adaptación al snapshot final M3	10
Verificación	10
Paso 7 - Arrancar y probar todos los casos	10
Adaptación al snapshot final M3	10
Verificación	10
Paso 8 - Provocar errores a propósito	10
Adaptación al snapshot final M3	11
Verificación	11
Paso 9 - Escribir tests del servicio	11
Adaptación al snapshot final M3	12
Verificación	12
Paso 10 - Escribir tests del controlador	12
Adaptación al snapshot final M3	13
Verificación	13
Paso 11 - Errores comunes del ejercicio	13
Adaptación al snapshot final M3	13
Verificación	13
Paso 12 - Reto resuelto - Filtro por rango de fechas	14
Adaptación al snapshot final M3	15
Verificación	15
Resultado esperado de la práctica	15
Resumen técnico del ejercicio	15
Pregunta final	15
Respuesta razonada	15
Práctica 3.2 - POST: crear recursos	15
Paso 1 - Repasar el estado actual	15
Adaptación al snapshot final M3	16
Verificación	16
Paso 2 - Mejorar la construcción de la URI con ServletUriComponentsBuilder	16
Adaptación al snapshot final M3	17

Verificación	17
Paso 3 - Arrancar y probar el POST con Location	17
Adaptación al snapshot final M3	17
Verificación	17
Paso 4 - Probar el POST con DNI duplicado	17
Adaptación al snapshot final M3	18
Verificación	18
Paso 5 - Probar el POST con JSON mal formado	18
Adaptación al snapshot final M3	18
Verificación	18
Paso 6 - Probar el POST sin Content-Type	18
Adaptación al snapshot final M3	19
Verificación	19
Paso 7 - Escribir tests del servicio para POST	19
Adaptación al snapshot final M3	19
Verificación	20
Paso 8 - Escribir tests del controlador para POST	20
Adaptación al snapshot final M3	21
Verificación	21
Paso 9 - Ejecutar todos los tests	21
Adaptación al snapshot final M3	21
Verificación	21
Paso 10 - Probar el POST con un cuerpo vacío	21
Adaptación al snapshot final M3	21
Verificación	22
Paso 11 - Errores comunes del ejercicio	22
Adaptación al snapshot final M3	22
Verificación	22
Paso 12 - Reto resuelto - POST con validación de campos obligatorios	22
Adaptación al snapshot final M3	23
Verificación	23
Resultado esperado de la práctica	23
Resumen técnico del ejercicio	23
Pregunta final	24
Respuesta razonada	24
Práctica 3.3 - PUT y PATCH: actualizar recursos	24
Paso 1 - Repasar el estado actual	24
Adaptación al snapshot final M3	25
Verificación	25
Paso 2 - Añadir validación de DNI duplicado en PUT	25
Adaptación al snapshot final M3	26
Verificación	26
Paso 3 - Probar el PUT con DNI duplicado	26
Adaptación al snapshot final M3	26
Verificación	26
Paso 4 - Probar el PUT sobre recurso inexistente	26
Adaptación al snapshot final M3	27
Verificación	27
Paso 5 - Añadir más campos al PATCH	27
Adaptación al snapshot final M3	28
Verificación	28
Paso 6 - Probar el PATCH con varios campos	28
Adaptación al snapshot final M3	28
Verificación	28
Paso 7 - Probar el PATCH con DNI duplicado	28
Adaptación al snapshot final M3	28

Verificación	28
Paso 8 - Probar el PATCH con fecha mal formateada	29
Adaptación al snapshot final M3	29
Verificación	29
Paso 9 - Escribir tests del servicio para PUT	29
Adaptación al snapshot final M3	30
Verificación	30
Paso 10 - Escribir tests del controlador para PUT y PATCH	30
Adaptación al snapshot final M3	31
Verificación	31
Paso 11 - Errores comunes del ejercicio	31
Adaptación al snapshot final M3	31
Verificación	32
Paso 12 - Reto resuelto - Actualizar solo el estado de un expediente con PATCH	32
Adaptación al snapshot final M3	32
Verificación	32
Resultado esperado de la práctica	32
Resumen técnico del ejercicio	33
Pregunta final	33
Respuesta razonada	33
Práctica 3.4 - DELETE: eliminar recursos	33
Paso 1 - Repasar el estado actual	33
Adaptación al snapshot final M3	34
Verificación	34
Paso 2 - Verificar el comportamiento esperado	34
Adaptación al snapshot final M3	34
Verificación	34
Paso 3 - Añadir un segundo recurso para probar dependencias	34
Adaptación al snapshot final M3	35
Verificación	35
Paso 4 - Añadir validación de dependencias en el servicio	35
Adaptación al snapshot final M3	36
Verificación	36
Paso 5 - Probar la eliminación con dependencias	36
Adaptación al snapshot final M3	36
Verificación	36
Paso 6 - Probar la eliminación sin dependencias	36
Adaptación al snapshot final M3	37
Verificación	37
Paso 7 - Probar la eliminación en cascada	37
Adaptación al snapshot final M3	37
Verificación	38
Paso 8 - Escribir tests del servicio para DELETE	38
Adaptación al snapshot final M3	38
Verificación	38
Paso 9 - Escribir tests del controlador para DELETE	39
Adaptación al snapshot final M3	39
Verificación	39
Paso 10 - Probar la idempotencia de DELETE	39
Adaptación al snapshot final M3	40
Verificación	40
Paso 11 - Errores comunes del ejercicio	40
Adaptación al snapshot final M3	40
Verificación	40
Paso 12 - Reto resuelto - Soft delete de alumnos	41
Adaptación al snapshot final M3	42

Verificación	42
Resultado esperado de la práctica	42
Resumen técnico del ejercicio	42
Pregunta final	42
Respuesta razonada	42
Práctica 3.5 - DTOs avanzados y validaciones	42
Paso 1 - Añadir la dependencia de validación	42
Adaptación al snapshot final M3	43
Verificación	43
Paso 2 - Crear el DTO de entrada AlumnoRequestDTO	43
Adaptación al snapshot final M3	44
Verificación	44
Paso 3 - Crear el DTO de salida AlumnoResponseDTO	44
Adaptación al snapshot final M3	45
Verificación	45
Paso 4 - Modificar el controlador para usar los nuevos DTOs	46
Adaptación al snapshot final M3	47
Verificación	47
Paso 5 - Añadir el manejador de errores de validación	47
Adaptación al snapshot final M3	47
Verificación	47
Paso 6 - Arrancar y probar el POST con datos válidos	48
Adaptación al snapshot final M3	48
Verificación	48
Paso 7 - Probar el POST con datos inválidos	48
Adaptación al snapshot final M3	48
Verificación	48
Paso 8 - Probar el POST sin fecha de nacimiento	49
Adaptación al snapshot final M3	49
Verificación	49
Paso 9 - Probar el POST con DNI duplicado	49
Adaptación al snapshot final M3	49
Verificación	49
Paso 10 - Escribir tests para validaciones	49
Adaptación al snapshot final M3	50
Verificación	50
Paso 11 - Errores comunes del ejercicio	50
Adaptación al snapshot final M3	51
Verificación	51
Paso 12 - Reto resuelto - DTO anidado para Expediente	51
Adaptación al snapshot final M3	53
Verificación	53
Resultado esperado de la práctica	53
Resumen técnico del ejercicio	53
Pregunta final	53
Respuesta razonada	53
Práctica 3.6 - Parámetros, cabeceras y Postman	53
Paso 1 - Añadir un endpoint que lea cabeceras	53
Adaptación al snapshot final M3	54
Verificación	54
Paso 2 - Probar con cabeceras personalizadas	54
Adaptación al snapshot final M3	55
Verificación	55
Paso 3 - Añadir un endpoint que lea cookies	55
Adaptación al snapshot final M3	55

Verificación	55
Paso 4 - Añadir un endpoint que acepte varios valores en un parámetro	55
Adaptación al snapshot final M3	56
Verificación	56
Paso 5 - Instalar Postman	56
Adaptación al snapshot final M3	56
Verificación	56
Paso 6 - Crear una colección en Postman	56
Adaptación al snapshot final M3	57
Verificación	57
Paso 7 - Crear un entorno en Postman	57
Adaptación al snapshot final M3	57
Verificación	57
Paso 8 - Crear una petición GET en Postman	57
Adaptación al snapshot final M3	57
Verificación	57
Paso 9 - Crear una petición POST en Postman	57
Adaptación al snapshot final M3	58
Verificación	58
Paso 10 - Añadir un test en Postman	58
Adaptación al snapshot final M3	58
Verificación	58
Paso 11 - Errores comunes del ejercicio	58
Adaptación al snapshot final M3	59
Verificación	59
Paso 12 - Reto resuelto - Endpoint que devuelve información completa de la petición	59
Adaptación al snapshot final M3	60
Verificación	60
Resultado esperado de la práctica	60
Resumen técnico del ejercicio	60
Pregunta final	61
Respuesta razonada	61

M3 - Parte práctica

Todas las prácticas parten del estado final de M2 y modifican el mismo proyecto. Antes de comenzar, ejecuta la suite heredada y comprueba que los endpoints acumulados siguen respondiendo. Cada paso conserva el propósito del PDF fuente y se enlaza con el snapshot final incluido en M3/proyecto.

Recorridos de entorno

Consola M3

Ejecuta `./mvnw` en Linux/macOS o `mvnw.cmd` en Windows desde M3/proyecto. Usa `curl -i` para observar status, cabeceras y cuerpo.

IntelliJ IDEA M3

Importa el `pom.xml` como proyecto Maven, usa Java 17, ejecuta `MiProyectoApplication` y los tests desde el IDE sin cambiar el contrato del proyecto.

Eclipse M3

Importa como Existing Maven Project, selecciona Java 17 y ejecuta la aplicación o los tests Maven conservando los mismos perfiles y propiedades.

VS Code M3

Abre M3/proyecto, usa Extension Pack for Java/Spring Boot y ejecuta el Maven Wrapper integrado en el proyecto.

Práctica 3.1 - GET: consultar recursos

Objetivo: aplicar de forma incremental el contrato de get: consultar recursos y conservar toda la funcionalidad heredada.

Paso 1 - Repasar el estado actual

Abre el AlumnoController y observa los endpoints GET actuales:

```
@GetMapping
public List<AlumnoDTO> listar(@RequestParam(required = false) String curso) {
    if (curso != null && !curso.isBlank()) {
        return service.listarPorCurso(curso);
    }
    return service.listar();
}

@GetMapping("/{id}")
public ResponseEntity<AlumnoDTO> consultar(@PathVariable String id) {
    return service.consultar(id)
        .map(ResponseEntity::ok)
        .orElse(ResponseEntity.notFound().build());
}
```

El GET de colección acepta un filtro por curso. El GET individual devuelve 200 o 404. Arranca la aplicación y prueba:

```
curl http://localhost:8080/api/v1/alumnos
curl http://localhost:8080/api/v1/alumnos/1
curl http://localhost:8080/api/v1/alumnos/999
```

Pregunta: ¿Qué crees que devuelve el GET de colección si no hay alumnos? ¿Y si el curso no existe?

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduces una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 2 - Ampliar el filtrado en el servicio

Vamos a ampliar el método listar del servicio para aceptar varios filtros: curso y DNI. Modifica AlumnoService:

```
public List<AlumnoDTO> listar(String curso, String dni) {
    return repositorio.listarTodos().stream()
        .filter(a -> curso == null || curso.isBlank()
```

```

        || a.getCurso().equalsIgnoreCase(curso))
    .filter(a -> dni == null || dni.isBlank()
        || a.getDni().equals(dni))
    .toList();
}

```

.filter(a -> curso == null || curso.isBlank() || ...) aplica el filtro solo si curso no es nulo ni vacío. Si es nulo o vacío, la

condición es true y no se filtra. .filter(a -> dni == null || dni.isBlank() || ...) hace lo mismo con el DNI. Los filtros se aplican en cadena. Si ambos son nulos, no se filtra por ninguno.

Pregunta: ¿Por qué es importante que los filtros sean condicionales? ¿Qué pasaría si no lo fueran?

Adaptación al snapshot final M3

Modifica AlumnoService para aceptar curso y dni. Los filtros sólo se aplican cuando el parámetro tiene valor.

Verificación

Comprueba **service.listar(curso, dni, sort, fechaDesde, fechaHasta, page, size)** y ejecuta **./mvnw test** (o **mvnw.cmd test** en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 3 - Añadir ordenación al servicio

Vamos a añadir ordenación al método listar. Modifica el método para aceptar un parámetro sort:

```

public List<AlumnoDTO> listar(String curso, String dni, String sort) {
    return repositorio.listarTodos().stream()
        .filter(a -> curso == null || curso.isBlank()
            || a.getCurso().equalsIgnoreCase(curso))
        .filter(a -> dni == null || dni.isBlank()
            || a.getDni().equals(dni))
        .sorted((a, b) -> {
            if ("nombre".equals(sort)) {
                return a.getNombre().compareToIgnoreCase(b.getNombre());
            }
            if ("apellidos".equals(sort)) {
                return a.getApellidos().compareToIgnoreCase(b.getApellidos());
            }
            if ("curso".equals(sort)) {
                return a.getCurso().compareToIgnoreCase(b.getCurso());
            }
            return 0;
        })
        .toList();
}

```

.sorted((a, b) -> ...) ordena el stream. El comparador comprueba qué campo se quiere ordenar y aplica el comparador

correspondiente. Si sort no coincide con ninguno de los campos, se devuelve 0, lo que deja el orden original.

Pregunta: ¿Qué orden tendría la lista si sort es null? ¿Y si es "dni"?

Adaptación al snapshot final M3

Centraliza el comparador para nombre, apellidos y curso. Un sort ausente conserva el orden del repositorio.

Verificación

Comprueba **sort=nombre** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 4 - Añadir paginación al servicio

Vamos a añadir paginación. Modifica el método para aceptar page y size:

```
public List<AlumnoDTO> listar(String curso, String dni, String sort, int page, int size) {
    return repositorio.listarTodos().stream()
        .filter(a -> curso == null || curso.isBlank()
            || a.getCurso().equalsIgnoreCase(curso))
        .filter(a -> dni == null || dni.isBlank()
            || a.getDni().equals(dni))
        .sorted((a, b) -> {
            if ("nombre".equals(sort)) {
                return a.getNombre().compareToIgnoreCase(b.getNombre());
            }
            if ("apellidos".equals(sort)) {
                return a.getApellidos().compareToIgnoreCase(b.getApellidos());
            }
            if ("curso".equals(sort)) {
                return a.getCurso().compareToIgnoreCase(b.getCurso());
            }
            return 0;
        })
        .skip((long) page * size)
        .limit(size)
        .toList();
}
```

`.skip((long) page * size)` descarta los primeros `page * size` elementos. `.limit(size)` limita el resultado a `size` elementos.

Pregunta: ¿Qué pasa si page es 0 y size es 10? ¿Y si page es 2 y size es 10?

Adaptación al snapshot final M3

Aplica `.skip((long) page * size)` y `.limit(size)` después de filtrar y ordenar. Valida `page` y `size` antes.

Verificación

Comprueba **page=1&size=2** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 5 - Añadir un método para contar elementos

Para construir los metadatos de paginación, necesitamos saber cuántos elementos hay en total (sin paginar). Añade un método al servicio:

```
public long contar(String curso, String dni) {
    return repositorio.listarTodos().stream()
        .filter(a -> curso == null || curso.isBlank()
            || a.getCurso().equalsIgnoreCase(curso))
        .filter(a -> dni == null || dni.isBlank()
            || a.getDni().equals(dni))
        .count();
}
```

.count() devuelve el número de elementos que cumplen los filtros. Es un long.

Pregunta: ¿Por qué el conteo no aplica paginación? ¿Qué representaría ese número?

Adaptación al snapshot final M3

Añade contar(...) usando exactamente los mismos filtros, sin paginación.

Verificación

Comprueba **totalElements** y **totalPages** y ejecuta ./mvnw test (o mvnw.cmd test en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 6 - Refactorizar el controlador para usar los nuevos parámetros

Modifica el método listar del controlador:

```
@GetMapping
public Map<String, Object> listar(
    @RequestParam(required = false) String curso,
    @RequestParam(required = false) String dni,
    @RequestParam(required = false) String sort,
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "20") int size) {

    List<AlumnoDTO> contenido = service.listar(curso, dni, sort, page, size);
    long total = service.contar(curso, dni);
    int totalPages = (int) Math.ceil((double) total / size);

    return Map.of(
        "content", contenido,
        "page", page,
        "size", size,
        "totalElements", total,
        "totalPages", totalPages
    );
}
```

Los parámetros curso, dni y sort son opcionales. page y size tienen valores por defecto. service.listar(...) devuelve la página actual. service.contar(...) devuelve el total de elementos que cumplen los filtros. Math.ceil((double) total / size) calcula el número total de páginas. Se redondea hacia arriba porque una página parcial cuenta como una página completa. Map.of(...) construye la respuesta con los metadatos. Spring Boot lo serializa a JSON.

Pregunta: ¿Por qué el método devuelve un Map en lugar de una lista? ¿Qué ventaja tiene?

Adaptación al snapshot final M3

El GET de colección devuelve content, page, size, totalElements y totalPages y valida $0 \leq \text{page}$, $1 \leq \text{size} \leq 100$.

Verificación

Comprueba **GET /api/v1/alumnos** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 7 - Arrancar y probar todos los casos

Arranca la aplicación y prueba:

```
# Listar todos (página 0, tamaño 20)
curl http://localhost:8080/api/v1/alumnos

# Listar con filtro por curso
curl "http://localhost:8080/api/v1/alumnos?curso=5º Primaria"

# Listar con ordenación por nombre
curl "http://localhost:8080/api/v1/alumnos?sort=nombre"

# Listar con paginación (página 0, tamaño 1)
curl "http://localhost:8080/api/v1/alumnos?page=0&size=1"

# Listar con paginación (página 1, tamaño 1)
curl "http://localhost:8080/api/v1/alumnos?page=1&size=1"

# Combinar filtro, ordenación y paginación
curl "http://localhost:8080/api/v1/alumnos?curso=5º Primaria&sort=nombre&page=0&size=10"

# Consultar individual existente
curl -i http://localhost:8080/api/v1/alumnos/1

# Consultar individual inexistente
curl -i http://localhost:8080/api/v1/alumnos/999
```

Pregunta: ¿Qué devuelve el GET de colección cuando el filtro no coincide con ningún alumno? ¿Es correcto?

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduces una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 8 - Provocar errores a propósito

Vamos a provocar algunos errores para entender cómo responde el servidor. Error 1: parámetro page negativo.

```
curl -i "http://localhost:8080/api/v1/alumnos?page=-1&size=10"
```

¿Qué crees que pasará? En nuestro caso, `page * size` sería negativo y `.skip(-10)` lanzaría `IllegalArgumentException`. Sin una validación previa, Spring Boot terminaría devolviendo un 500.

Error 2: parámetro size cero.

```
curl -i "http://localhost:8080/api/v1/alumnos?page=0&size=0"
```

`.limit(0)` devuelve una lista vacía. No es un error, pero puede confundir. En una API real, se validaría que size sea positivo.

Error 3: tipo incorrecto en un parámetro.

```
curl -i "http://localhost:8080/api/v1/alumnos?page=abc"
```

Spring MVC intenta convertir "abc" a int y falla. Devuelve un 400 Bad Request.

Pregunta: ¿Qué diferencia hay entre un 400 y un 500? ¿Cuál es culpa del cliente y cuál del servidor?

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduces una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 9 - Escribir tests del servicio

Vamos a escribir tests para el método listar del servicio. Crea `AlumnoServiceTest` si no lo tienes, y añade tests:

```
@Test
void listar_debeDevolverTodos_cuandoNoHayFiltros() {
    when(repositorio.listarTodos()).thenReturn(List.of(
        new AlumnoDTO("1", "Ana", "García", "12345678A", null, "5º"),
        new AlumnoDTO("2", "Luis", "Pérez", "87654321B", null, "6º")
    ));

    List<AlumnoDTO> resultado = servicio.listar(null, null, null, 0, 20);

    assertEquals(2, resultado.size());
}

@Test
void listar_debeFiltrarPorCurso() {
    when(repositorio.listarTodos()).thenReturn(List.of(
        new AlumnoDTO("1", "Ana", "García", "12345678A", null, "5º"),
        new AlumnoDTO("2", "Luis", "Pérez", "87654321B", null, "6º")
    ));

    List<AlumnoDTO> resultado = servicio.listar("5º", null, null, 0, 20);

    assertEquals(1, resultado.size());
    assertEquals("Ana", resultado.get(0).getNombre());
}
```

```

@Test
void listar_debeOrdenarPorNombre() {
    when(repositorio.listarTodos()).thenReturn(List.of(
        new AlumnoDTO("2", "Luis", "Pérez", "87654321B", null, "6º"),
        new AlumnoDTO("1", "Ana", "García", "12345678A", null, "5º")
    ));

    List<AlumnoDTO> resultado = servicio.listar(null, null, "nombre", 0, 20);

    assertEquals("Ana", resultado.get(0).getNombre());
    assertEquals("Luis", resultado.get(1).getNombre());
}

@Test
void listar_debePaginar() {
    when(repositorio.listarTodos()).thenReturn(List.of(
        new AlumnoDTO("1", "Ana", "García", "12345678A", null, "5º"),
        new AlumnoDTO("2", "Luis", "Pérez", "87654321B", null, "6º"),
        new AlumnoDTO("3", "María", "López", "11111111C", null, "4º")
    ));

    List<AlumnoDTO> resultado = servicio.listar(null, null, null, 1, 2);

    assertEquals(1, resultado.size());
    assertEquals("María", resultado.get(0).getNombre());
}

```

Pregunta: ¿Por qué el test de paginación verifica que el tamaño del resultado es 1? ¿Qué representa ese 1?

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduces una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 10 - Escribir tests del controlador

Añade tests al `AlumnoControllerTest`:

```

@Test
void listar_debeDevolver200ConMetadatos() throws Exception {
    when(service.listar(any(), any(), any(), anyInt(), anyInt()))
        .thenReturn(List.of(new AlumnoDTO("1", "Ana", "García", "12345678A", null, "5º")));
    when(service.contar(any(), any())).thenReturn(1L);

    mockMvc.perform(get("/api/v1/alumnos"))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.content").isArray())
        .andExpect(jsonPath("$.content.length()").value(1))
        .andExpect(jsonPath("$.page").value(0))

```

```

    .andExpect(jsonPath("$.size").value(20))
    .andExpect(jsonPath("$.totalElements").value(1))
    .andExpect(jsonPath("$.totalPages").value(1));
}

```

any() es un matcher de Mockito que coincide con cualquier valor, incluido null. anyInt() coincide con cualquier int. `jsonPath("$.content").isArray().verificaquecontentesunarray.jsonPath("$.content.length()").value(1)` verifica que el array tiene 1 elemento. `jsonPath("$.page").value(0)` verifica que page es 0.

Pregunta: ¿Por qué se mockean listar y contar con any() en lugar de con valores concretos?

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduzcas una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 11 - Errores comunes del ejercicio

Error	Causa	Solución
500 Internal Server Error con page negativo	<code>.skip()</code> recibe un valor negativo	Validar <code>page >= 0</code> antes de paginar
400 Bad Request con <code>page=abc</code>	Spring no puede convertir <code>abc</code> a int	Documentar que <code>page</code> debe ser numérico
La colección vacía devuelve 404	Se trata como recurso no encontrado	Devolver [] con 200 OK
El filtro no se aplica	Se compara con <code>==</code> en lugar de <code>.equals()</code>	Usar <code>.equals()</code> o <code>equalsIgnoreCase()</code>
La ordenación no funciona	El comparador no contempla el campo	Definir un comportamiento explícito para campos desconocidos
<code>totalPages</code> es 0	<code>totalElements</code> es 0	Es correcto: cero elementos implican cero páginas
El test falla por <code>any()</code>	Se mezclan matchers con valores concretos	Usar matchers en todos los argumentos
La paginación no devuelve datos	<code>page</code> y <code>size</code> están mal calculados	Revisar <code>skip</code> y <code>limit</code>

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduzcas una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 12 - Reto resuelto - Filtro por rango de fechas

Reto: Añadir un filtro por rango de fecha de nacimiento. El cliente envía fechaDesde y fechaHasta como query parameters, y el servidor devuelve solo los alumnos nacidos en ese rango. Solución paso a paso: Paso 1: Añadir los parámetros al método listar del servicio:

```
public List<AlumnoDTO> listar(String curso, String dni, String sort,
                             LocalDate fechaDesde, LocalDate fechaHasta,
                             int page, int size) {
    return repositorio.listarTodos().stream()
        .filter(a -> curso == null || curso.isBlank()
            || a.getCurso().equalsIgnoreCase(curso))
        .filter(a -> dni == null || dni.isBlank()
            || a.getDni().equals(dni))
        .filter(a -> fechaDesde == null
            || (a.getFechaNacimiento() != null
                && !a.getFechaNacimiento().isBefore(fechaDesde)))
        .filter(a -> fechaHasta == null
            || (a.getFechaNacimiento() != null
                && !a.getFechaNacimiento().isAfter(fechaHasta)))
        .sorted(...)
        .skip((long) page * size)
        .limit(size)
        .toList();
}
```

!a.getFechaNacimiento().isBefore(fechaDesde) significa “la fecha de nacimiento no es anterior a fechaDesde”, es decir, es igual o

posterior. !a.getFechaNacimiento().isAfter(fechaHasta) significa “la fecha de nacimiento no es posterior a fechaHasta”, es decir, es igual o anterior. Paso 2: Modificar el controlador para aceptar los nuevos parámetros:

```
@GetMapping
public Map<String, Object> listar(
    @RequestParam(required = false) String curso,
    @RequestParam(required = false) String dni,
    @RequestParam(required = false) String sort,
    @RequestParam(required = false)
    @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate fechaDesde,
    @RequestParam(required = false)
    @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate fechaHasta,
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "20") int size) {
    // ...
}
```

@DateTimeFormat(iso = DateTimeFormat.ISO.DATE) indica a Spring que convierta el parámetro de String a LocalDate usando el formato ISO-8601.

formato ISO 8601 (yyyy-MM-dd). Paso 3: Probar:

```
curl "http://localhost:8080/api/v1/alumnos?fechaDesde=2010-01-01&fechaHasta=2010-12-31"
```

Devuelve los alumnos nacidos en 2010.

Pregunta: ¿Qué formato de fecha espera el parámetro fechaDesde? ¿Qué pasaría si se envía en otro formato?

Adaptación al snapshot final M3

Añade `fechaDesde` y `fechaHasta` como `LocalDate` ISO, inclusivas, y rechaza un rango invertido.

Verificación

Comprueba `fechaDesde=2010-01-01&fechaHasta=2010-12-31` y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Resultado esperado de la práctica

Al final del ejercicio, deberías tener:

- Un GET de colección que acepta filtros (`curso`, `dni`, `fechaDesde`, `fechaHasta`), ordenación (`sort`) y paginación (`page`, `size`).
- Un GET individual que devuelve 200 o 404.
- Una respuesta paginada con metadatos (`content`, `page`, `size`, `totalElements`, `totalPages`).
- Tests del servicio y del controlador para los casos principales.

Resumen técnico del ejercicio

El ejercicio ha demostrado:

- GET de colección: filtrado, ordenación y paginación.
- GET individual: 200 o 404.
- `@PathVariable` y `@RequestParam`: cuándo usar cada uno.
- Streams: `.filter()`, `.sorted()`, `.skip()`, `.limit()`, `.count()`.
- Optional: para resultados únicos.
- Metadatos de paginación: `content`, `page`, `size`, `totalElements`, `totalPages`.
- Tests: servicio con Mockito, controlador con MockMvc.
- Errores provocados: `page` negativo, `size` cero, tipo incorrecto.

Pregunta final

¿Qué evidencia mínima demuestra que la práctica 3.1 quedó integrada sin romper M2?

Respuesta razonada

La suite completa debe seguir verde y, además, una petición positiva y otra negativa representativas del punto deben producir el status y el cuerpo previstos. Esa combinación verifica regresión y comportamiento nuevo.

Práctica 3.2 - POST: crear recursos

Objetivo: aplicar de forma incremental el contrato de post: crear recursos y conservar toda la funcionalidad heredada.

Paso 1 - Repasar el estado actual

Abre el `AlumnoController` y observa el método `crear` actual:

```
@PostMapping
public ResponseEntity<AlumnoDTO> crear(@RequestBody AlumnoDTO dto) {
    AlumnoDTO creado = service.crear(dto);
    URI location = URI.create("/api/v1/alumnos/" + creado.getIdificador());
```

```
    return ResponseEntity.created(location).body(creado);
}
```

Y el método crear del AlumnoService:

```
public AlumnoDTO crear(AlumnoDTO dto) {
    if (repositorio.existePorDni(dto.getDni())) {
        throw new NegocioException("Ya existe un alumno con el DNI " + dto.getDni());
    }
    if (dto.getIdificador() == null || dto.getIdificador().isBlank()) {
        dto.setIdificador(String.valueOf(repositorio.contar() + 1));
    }
    return repositorio.guardar(dto);
}
```

Arranca la aplicación y prueba:

```
curl -i -X POST http://localhost:8080/api/v1/alumnos \
-H "Content-Type: application/json" \
-d '{"nombre":"María","apellidos":"López","dni":"33333333E","fechaNacimiento":"2011-03-20","curso":1}'
```

Verás un 201 con la cabecera Location y el recurso creado.

Pregunta: ¿Qué pasaría si el DNI ya existiera? ¿Qué código devolvería?

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduces una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 2 - Mejorar la construcción de la URI con ServletUriComponents-Builder

Actualmente construimos la URI concatenando strings:

```
URI location = URI.create("/api/v1/alumnos/" + creado.getIdificador());
```

Vamos a mejorarlo con ServletUriComponentsBuilder, que es más robusto:

```
import org.springframework.web.servlet.support.ServletUriComponentsBuilder;

@PostMapping
public ResponseEntity<AlumnoDTO> crear(@RequestBody AlumnoDTO dto) {
    AlumnoDTO creado = service.crear(dto);
    URI location = ServletUriComponentsBuilder
        .fromCurrentRequest()
        .path("/{id}")
        .buildAndExpand(creado.getIdificador())
        .toUri();
    return ResponseEntity.created(location).body(creado);
}
```

`fromCurrentRequest()` toma la URL actual (`/api/v1/alumnos`). `.path("/{id}")` añade la ruta del recurso individual. `.buildAndExpand(id)` reemplaza `{id}` con el ID del recurso creado. `.toUri()` con-

struye la URI final. Esta forma es más robusta: si mañana cambia el contexto de la aplicación (por ejemplo, se despliega en /mecd/api), la URI se construye correctamente.

Pregunta: ¿Qué ventaja tiene ServletUriComponentsBuilder frente a concatenar strings?

Adaptación al snapshot final M3

Sustituye la concatenación manual de Location por `ServletUriComponentsBuilder.fromCurrentRequest().path`

Verificación

Comprueba **Location**: `http://localhost:8080/api/v1/alumnos/{id}` y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 3 - Arrancar y probar el POST con Location

Reinicia la aplicación y prueba:

```
curl -i -X POST http://localhost:8080/api/v1/alumnos \
  -H "Content-Type: application/json" \
  -d '{"nombre":"Pedro","apellidos":"Sánchez","dni":"44444444F","fechaNacimiento":"2010-07-15","cu
```

Verás algo como:

```
HTTP/1.1 201
Location: http://localhost:8080/api/v1/alumnos/3
Content-Type: application/json
```

```
{"id":"3","nombre":"Pedro",...}
```

La cabecera Location incluye la URL completa. El cliente puede usarla para consultar el recurso.

Pregunta: ¿Por qué la cabecera Location incluye el host y el puerto, no solo la ruta?

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduces una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 4 - Probar el POST con DNI duplicado

Ahora prueba a crear un alumno con un DNI que ya existe:

```
curl -i -X POST http://localhost:8080/api/v1/alumnos \
  -H "Content-Type: application/json" \
  -d '{"nombre":"Otro","apellidos":"Alumno","dni":"12345678A","fechaNacimiento":"2010-01-01","curs
```

Verás algo como:

```
HTTP/1.1 409
Content-Type: application/json
{"status":409,"error":"Conflict","message":"Ya existe un alumno con el DNI 12345678A"}
```

El servicio ha detectado el duplicado y ha lanzado `NegocioException`. El manejador del controlador la captura y devuelve 409 con un JSON descriptivo.

Pregunta: ¿Por qué es 409 y no 400? ¿Qué diferencia hay entre un conflicto y una petición mal formada?

Adaptación al snapshot final M3

Reutiliza la regla de DNI único del servicio. El conflicto debe quedar en 409 y no debe guardar nada.

Verificación

Comprueba **409 Conflict** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 5 - Probar el POST con JSON mal formado

Vamos a provocar un error de sintaxis en el JSON:

```
curl -i -X POST http://localhost:8080/api/v1/alumnos \
  -H "Content-Type: application/json" \
  -d '{"nombre":"Pedro", "apellidos":"Sánchez",}'
```

El JSON tiene una coma final. Verás un 400 Bad Request con un mensaje que describe el error:

```
{
  "timestamp": "...",
  "status": 400,
  "error": "Bad Request",
  "message": "JSON parse error: Unexpected character (',' (code 44)): expected a valid value"
}
```

Spring Boot captura la excepción de Jackson y devuelve un 400 con el detalle. El cliente sabe exactamente qué ha fallado.

Pregunta: ¿Qué otros errores de formato provocarían un 400?

Adaptación al snapshot final M3

Envía JSON con una coma sobrante para observar un 400 de deserialización antes de la lógica de negocio.

Verificación

Comprueba **400 Bad Request** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 6 - Probar el POST sin Content-Type

Vamos a enviar un POST sin la cabecera Content-Type:

```
curl -i -X POST http://localhost:8080/api/v1/alumnos \
  -d '{"nombre":"Pedro","apellidos":"Sánchez","dni":"55555555G","fechaNacimiento":"2010-01-01","cu"
```

Verás un 415 Unsupported Media Type. Spring MVC no sabe cómo deserializar el cuerpo sin conocer el tipo. El cliente debe enviar Content-Type: application/json.

Pregunta: ¿Por qué Spring MVC necesita la cabecera Content-Type para deserializar?

Adaptación al snapshot final M3

Envía el mismo cuerpo sin Content-Type: application/json y observa el rechazo del media type.

Verificación

Comprueba **415 Unsupported Media Type** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 7 - Escribir tests del servicio para POST

Vamos a escribir tests para el método crear del servicio. Crea o amplía `AlumnoServiceTest`:

```
@Test
void crear_debeGuardarYDevolverAlumno_cuandoNoDuplicado() {
    AlumnoDTO dto = new AlumnoDTO();
    dto.setNombre("María");
    dto.setApellidos("López");
    dto.setDni("11111111C");
    dto.setCurso("4º Primaria");

    when(repositorio.existePorDni("11111111C")).thenReturn(false);
    when(repositorio.contar()).thenReturn(5);
    when(repositorio.guardar(any(AlumnoDTO.class)))
        .thenAnswer(inv -> inv.getArgument(0));

    AlumnoDTO resultado = servicio.crear(dto);

    assertNotNull(resultado);
    assertEquals("6", resultado.getIdentificador());
    verify(repositorio).guardar(any(AlumnoDTO.class));
}

@Test
void crear_debeLanzarExcepcion_cuandoDniDuplicado() {
    AlumnoDTO dto = new AlumnoDTO();
    dto.setDni("12345678A");

    when(repositorio.existePorDni("12345678A")).thenReturn(true);

    assertThrows(NegocioException.class, () -> servicio.crear(dto));
    verify(repositorio, never()).guardar(any(AlumnoDTO.class));
}
```

`verify(repositorio, never()).guardar(...)` verifica que no se ha llamado a guardar. Eso es importante: si el DNI está duplicado, no

se debe guardar nada.

Pregunta: ¿Por qué es importante verificar que no se ha llamado a guardar cuando hay duplicado?

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduces una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 8 - Escribir tests del controlador para POST

Añade tests al `AlumnoControllerTest`:

```
@Test
void crear_debeDevolver201_cuandoDatosValidos() throws Exception {
    AlumnoDTO dto = new AlumnoDTO("3", "María", "López", "11111111C",
        LocalDate.of(2011, 3, 20), "4º Primaria");
    when(service.crear(any(AlumnoDTO.class))).thenReturn(dto);

    mockMvc.perform(post("/api/v1/alumnos")
        .contentType(MediaType.APPLICATION_JSON)
        .content("""
            {
                "nombre": "María",
                "apellidos": "López",
                "dni": "11111111C",
                "fechaNacimiento": "2011-03-20",
                "curso": "4º Primaria"
            }
            """))
        .andExpect(status().isCreated())
        .andExpect(header().exists("Location"))
        .andExpect(jsonPath("$.id").value("3"))
        .andExpect(jsonPath("$.nombre").value("María"));
}

@Test
void crear_debeDevolver409_cuandoDniDuplicado() throws Exception {
    when(service.crear(any(AlumnoDTO.class)))
        .thenThrow(new NegocioException("Ya existe un alumno con el DNI 12345678A"));

    mockMvc.perform(post("/api/v1/alumnos")
        .contentType(MediaType.APPLICATION_JSON)
        .content("""
            {
                "nombre": "Otro",
                "apellidos": "Alumno",
                "dni": "12345678A",
                "fechaNacimiento": "2010-01-01",
                "curso": "1º Primaria"
            }
            """))
        .andExpect(status().isConflict())
        .andExpect(jsonPath("$.status").value(409));
}
```

`header().exists("Location")` verifica que la cabecera Location existe, sin comprobar su valor exacto. Es útil cuando la URI completa

depende del host. `thenThrow(new NegocioException(...))` configura el mock para que lance la excepción. Así el controlador la captura con `@ExceptionHandler` y devuelve 409.

Pregunta: ¿Por qué el test del controlador mockea el servicio en lugar de usar uno real?

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduces una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 9 - Ejecutar todos los tests

Ejecuta los tests desde la terminal:

```
./mvnw test
```

Verás un resumen con los tests que pasan y los que fallan. Si todo va bien:

```
Tests run: 12, Failures: 0, Errors: 0, Skipped: 0
```

Pregunta: ¿Qué ventaja tiene ejecutar todos los tests antes de subir un cambio?

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduces una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 10 - Probar el POST con un cuerpo vacío

Vamos a provocar un error enviando un POST sin cuerpo:

```
curl -i -X POST http://localhost:8080/api/v1/alumnos \
-H "Content-Type: application/json"
```

Verás un 400 Bad Request con un mensaje como:

```
{
  "status": 400,
  "error": "Bad Request",
  "message": "Required request body is missing"
}
```

Spring MVC detecta que falta el cuerpo y devuelve 400. Es un error del cliente, no del servidor.

Pregunta: ¿Qué diferencia hay entre “cuerpo vacío” y “cuerpo con JSON mal formado”? ¿Ambos devuelven 400?

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduces una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 11 - Errores comunes del ejercicio

Error	Causa	Solución
415 Unsupported Media Type	Falta Content-Type	Añadir -H "Content-Type: application/json"
400 Bad Request	JSON mal formado	Revisar la sintaxis del cuerpo
400 Bad Request con cuerpo vacío	No se ha enviado un cuerpo	Enviar un JSON válido
409 Conflict	DNI duplicado	Es correcto: el servicio detecta el conflicto
500 Internal Server Error	Excepción no traducida	Añadir un manejador para la excepción conocida
No aparece Location	Falta ResponseEntity.created(location)	Construir la respuesta con <code>created(...)</code>
El ID no se asigna	El servicio no genera identidad	Revisar la lógica de creación
El test falla por <code>any()</code>	Matchers mezclados	Usar matchers en todos los argumentos
<code>MethodArgumentNotValidException</code> no se traduce	Falta el manejador	Añadirlo al manejo centralizado de errores

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduzcas una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 12 - Reto resuelto - POST con validación de campos obligatorios

Reto: Añadir validación manual en el servicio para comprobar que los campos nombre, apellidos y dni no estén vacíos. Si alguno lo está, lanzar `NegocioException` con un mensaje descriptivo. Solución paso a paso: Paso 1: Modificar el método crear del servicio:

```
public AlumnoDTO crear(AlumnoDTO dto) {
    if (dto.getNombre() == null || dto.getNombre().isBlank()) {
        throw new NegocioException("El nombre es obligatorio");
    }
    if (dto.getApellidos() == null || dto.getApellidos().isBlank()) {
        throw new NegocioException("Los apellidos son obligatorios");
    }
    if (dto.getDni() == null || dto.getDni().isBlank()) {
        throw new NegocioException("El DNI es obligatorio");
    }
    if (repositorio.existePorDni(dto.getDni())) {
        throw new NegocioException("Ya existe un alumno con el DNI " + dto.getDni());
    }
}
```

```

    }
    if (dto.getIdentificador() == null || dto.getIdentificador().isBlank()) {
        dto.setIdentificador(String.valueOf(repositorio.contar() + 1));
    }
    return repositorio.guardar(dto);
}

```

Cada validación comprueba que un campo no sea nulo ni vacío. Si falla, lanza `NegocioException` con un mensaje específico. Paso 2: Probar con curl:

```

# Sin nombre
curl -i -X POST http://localhost:8080/api/v1/alumnos \
-H "Content-Type: application/json" \
-d '{"apellidos":"López","dni":"66666666H","fechaNacimiento":"2010-01-01","curso":"1º"}'

# Sin DNI
curl -i -X POST http://localhost:8080/api/v1/alumnos \
-H "Content-Type: application/json" \
-d '{"nombre":"Pedro","apellidos":"López","fechaNacimiento":"2010-01-01","curso":"1º"}'

```

En esta fase transitoria verás el código definido por la validación manual. En el punto 3.5, al centralizar Bean Validation, quedará separada la validación de formato (400) del conflicto de negocio (409).

Pregunta: ¿Por qué hemos usado 409 y no 400 para los campos obligatorios? ¿Es correcto?

En realidad, los campos obligatorios son un error de formato (400), no un conflicto (409). Pero como todavía no hemos visto Bean Validation ni un manejador global, hemos usado el mismo 409 para simplificar. En el Módulo 5 lo corregiremos.

Adaptación al snapshot final M3

Introduce primero la validación manual sólo como transición pedagógica; en 3.5 se sustituirá por Bean Validation.

Verificación

Comprueba **nombre**, **apellidos**, **dni obligatorios** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Resultado esperado de la práctica

Al final del ejercicio, deberías tener:

- Un POST que devuelve 201 con Location y el recurso creado.
- Validación de negocio en el servicio (DNI duplicado, campos obligatorios).
- Manejo de `NegocioException` con 409.
- Tests del servicio y del controlador para POST.
- Capacidad de probar todos los casos con curl.

Resumen técnico del ejercicio

El ejercicio ha demostrado:

- POST: crear un recurso. 201 Created con Location.
- `@RequestBody`: deserializa el cuerpo a un DTO.
- `ServletUriComponentsBuilder`: construye la URI del recurso creado.

- Validación de negocio: en el servicio, con `NegocioException`.
- Manejo de errores: `@ExceptionHandler` para 409.
- Tests: servicio con `Mockito`, controlador con `MockMvc`.
- Errores provocados: JSON mal formado, sin `Content-Type`, cuerpo vacío, DNI duplicado.

Pregunta final

¿Qué evidencia mínima demuestra que la práctica 3.2 quedó integrada sin romper M2?

Respuesta razonada

La suite completa debe seguir verde y, además, una petición positiva y otra negativa representativas del punto deben producir el status y el cuerpo previstos. Esa combinación verifica regresión y comportamiento nuevo.

Práctica 3.3 - PUT y PATCH: actualizar recursos

Objetivo: aplicar de forma incremental el contrato de put y patch: actualizar recursos y conservar toda la funcionalidad heredada.

Paso 1 - Repasar el estado actual

Abre el `AlumnoController` y observa los métodos `actualizar` y `actualizarParcial`:

```
@PutMapping("/{id}")
public ResponseEntity<AlumnoDTO> actualizar(
    @PathVariable String id,
    @RequestBody AlumnoDTO dto) {
    return service.actualizar(id, dto)
        .map(ResponseEntity::ok)
        .orElse(ResponseEntity.notFound().build());
}
```

```
@PatchMapping("/{id}")
public ResponseEntity<AlumnoDTO> actualizarParcial(
    @PathVariable String id,
    @RequestBody Map<String, Object> cambios) {
    return service.actualizarParcial(id, cambios)
        .map(ResponseEntity::ok)
        .orElse(ResponseEntity.notFound().build());
}
```

Y los métodos del `AlumnoService`:

```
public Optional<AlumnoDTO> actualizar(String id, AlumnoDTO dto) {
    return repositorio.buscarPorId(id).map(existente -> {
        dto.setIdentificador(id);
        return repositorio.guardar(dto);
    });
}

public Optional<AlumnoDTO> actualizarParcial(String id, Map<String, Object> cambios) {
    return repositorio.buscarPorId(id).map(alumno -> {
        if (cambios.containsKey("nombre")) {
            alumno.setNombre((String) cambios.get("nombre"));
        }
        if (cambios.containsKey("apellidos")) {

```

```

        alumno.setApellidos((String) cambios.get("apellidos"));
    }
    if (cambios.containsKey("curso")) {
        alumno.setCurso((String) cambios.get("curso"));
    }
    return repositorio.guardar(alumno);
});
}

```

Arranca la aplicación y prueba:

```

# PUT
curl -i -X PUT http://localhost:8080/api/v1/alumnos/1 \
  -H "Content-Type: application/json" \
  -d '{
    "nombre": "Ana",
    "apellidos": "García López",
    "dni": "12345678A",
    "fechaNacimiento": "2010-05-12",
    "curso": "6º Primaria"
  }'

```

```

# PATCH
curl -i -X PATCH http://localhost:8080/api/v1/alumnos/1 \
  -H "Content-Type: application/json" \
  -d '{"curso": "6º Primaria B"}'

```

Pregunta: ¿Qué diferencia hay entre el PUT y el PATCH que acabas de probar?

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduces una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 2 - Añadir validación de DNI duplicado en PUT

Vamos a añadir validación de DNI duplicado en el método actualizar. Primero, añade el método al repositorio:

```

public boolean existePorDniYIdDistinto(String dni, String id) {
    return almacen.values().stream()
        .anyMatch(a -> a.getDni().equals(dni)
            && !a.getIdentificador().equals(id));
}

```

`anyMatch(a -> ...)` comprueba si algún elemento cumple la condición. `a.getDni().equals(dni)` comprueba que el DNI coincide. `!a.getIdentificador().equals(id)` comprueba que el ID es distinto. Así no se considera duplicado si el DNI pertenece al mismo alumno. Ahora modifica el método actualizar del servicio:

```

public Optional<AlumnoDTO> actualizar(String id, AlumnoDTO dto) {
    if (dto.getDni() != null

```

```

        && repositorio.existePorDniYIdDistinto(dto.getDni(), id)) {
            throw new NegocioException(
                "Ya existe otro alumno con el DNI " + dto.getDni());
        }
        return repositorio.buscarPorId(id).map(existente -> {
            dto.setIdentificador(id);
            return repositorio.guardar(dto);
        });
    }
}

```

`dto.getDni() != null` comprueba que el DNI no es nulo antes de validar. Si el cliente no envía DNI en el PUT, no se valida. `repositorio.existePorDniYIdDistinto(...)` comprueba si hay otro alumno con ese DNI. `throw new NegocioException(...)` lanza la excepción si hay duplicado.

Pregunta: ¿Por qué se comprueba que el DNI no es nulo antes de validar?

Adaptación al snapshot final M3

Antes de guardar un PUT, comprueba si el DNI pertenece a otro alumno. La identidad de la URL se conserva.

Verificación

Comprueba **existePorDniYIdDistinto o comprobación equivalente** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 3 - Probar el PUT con DNI duplicado

Reinicia la aplicación y prueba a actualizar un alumno con un DNI que ya existe en otro:

```

curl -i -X PUT http://localhost:8080/api/v1/alumnos/1 \
  -H "Content-Type: application/json" \
  -d '{"nombre":"Ana","apellidos":"García","dni":"87654321B","fechaNacimiento":"2010-05-12","curso":1}'

```

El DNI 87654321B pertenece al alumno 2. Como estamos actualizando el alumno 1, hay conflicto. Verás un 409 Conflict con el mensaje:

```

{"status":409,"error":"Conflict","message":"Ya existe otro alumno con el DNI 87654321B"}

```

Pregunta: ¿Qué pasaría si actualizáramos el alumno 2 con su propio DNI? ¿Habría conflicto?

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduces una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 4 - Probar el PUT sobre recurso inexistente

Prueba a actualizar un alumno que no existe:

```
curl -i -X PUT http://localhost:8080/api/v1/alumnos/999 \
  -H "Content-Type: application/json" \
  -d '{"nombre":"X","apellidos":"Y","dni":"00000000Z","fechaNacimiento":"2010-01-01","curso":"1º"}'
```

Verás un 404 Not Found. El servicio devuelve `Optional.empty()` porque no encuentra el recurso y el controlador lo traduce a 404.

Pregunta: ¿Por qué el 404 es el código correcto en este caso?

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduces una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 5 - Añadir más campos al PATCH

Vamos a ampliar el PATCH para que permita modificar más campos: dni, fechaNacimiento e importe. Modifica el método `actualizarParcial` del servicio:

```
public Optional<AlumnoDTO> actualizarParcial(String id, Map<String, Object> cambios) {
    return repositorio.buscarPorId(id).map(alumno -> {
        if (cambios.containsKey("nombre")) {
            alumno.setNombre((String) cambios.get("nombre"));
        }
        if (cambios.containsKey("apellidos")) {
            alumno.setApellidos((String) cambios.get("apellidos"));
        }
        if (cambios.containsKey("dni")) {
            String nuevoDni = (String) cambios.get("dni");
            if (repositorio.existePorDniYIdDistinto(nuevoDni, id)) {
                throw new NegocioException(
                    "Ya existe otro alumno con el DNI " + nuevoDni);
            }
            alumno.setDni(nuevoDni);
        }
        if (cambios.containsKey("curso")) {
            alumno.setCurso((String) cambios.get("curso"));
        }
        if (cambios.containsKey("fechaNacimiento")) {
            alumno.setFechaNacimiento(
                LocalDate.parse((String) cambios.get("fechaNacimiento")));
        }
        return repositorio.guardar(alumno);
    });
}
```

`LocalDate.parse(...)` convierte un String en formato ISO 8601 (yyyy-MM-dd) a un `LocalDate`. Si el formato no es correcto, lanza `DateTimeParseException`.

Pregunta: ¿Qué pasa si el cliente envía una fecha con formato incorrecto en el PATCH?

Adaptación al snapshot final M3

Amplía PATCH a nombre, apellidos, dni, curso y fechaNacimiento; ignora o rechaza campos no admitidos de forma explícita.

Verificación

Comprueba **Map<String, Object> cambios** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 6 - Probar el PATCH con varios campos

Reinicia y prueba un PATCH que modifique varios campos:

```
curl -i -X PATCH http://localhost:8080/api/v1/alumnos/1 \
  -H "Content-Type: application/json" \
  -d '{"nombre": "Ana María", "curso": "6º Primaria B"}'
```

Verás un 200 con el alumno actualizado. Solo han cambiado nombre y curso; los demás campos se mantienen.

Pregunta: ¿Cómo puedes verificar que los demás campos no han cambiado?

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduces una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 7 - Probar el PATCH con DNI duplicado

Prueba un PATCH que cambie el DNI a uno que ya existe:

```
curl -i -X PATCH http://localhost:8080/api/v1/alumnos/1 \
  -H "Content-Type: application/json" \
  -d '{"dni": "87654321B"}'
```

Verás un 409 Conflict. El servicio detecta el duplicado y lanza `NegocioException`.

Pregunta: ¿Qué diferencia hay entre este 409 y el del PUT?

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduces una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 8 - Probar el PATCH con fecha mal formateada

Prueba un PATCH con una fecha en formato incorrecto:

```
curl -i -X PATCH http://localhost:8080/api/v1/alumnos/1 \
  -H "Content-Type: application/json" \
  -d '{"fechaNacimiento": "12/05/2010"}'
```

LocalDate.parse espera formato ISO 8601 (yyyy-MM-dd). Como "12/05/2010" no lo es, lanza DateTimeParseException. Spring Boot devuelve un 500 Internal Server Error.

Pregunta: ¿Qué habría que hacer para aceptar también el formato dd/MM/yyyy?

Adaptación al snapshot final M3

Provoca una fecha no ISO en PATCH y asegúrate de obtener 400, nunca 500.

Verificación

Comprueba **fechaNacimiento=15/07/2010** y ejecuta ./mvnw test (o mvnw.cmd test en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 9 - Escribir tests del servicio para PUT

Añade tests al AlumnoServiceTest:

```
@Test
void actualizar_debeReemplazarAlumno_cuandoExiste() {
    AlumnoDTO existente = new AlumnoDTO("1", "Ana", "García", "12345678A",
        LocalDate.of(2010, 5, 12), "5º Primaria");
    AlumnoDTO nuevosDatos = new AlumnoDTO(null, "Ana", "García López", "12345678A",
        LocalDate.of(2010, 5, 12), "6º Primaria");

    when(repositorio.buscarPorId("1")).thenReturn(Optional.of(existente));
    when(repositorio.existePorDniYIdDistinto("12345678A", "1")).thenReturn(false);
    when(repositorio.guardar(any())).thenAnswer(inv -> inv.getArgument(0));

    Optional<AlumnoDTO> resultado = servicio.actualizar("1", nuevosDatos);

    assertTrue(resultado.isPresent());
    assertEquals("6º Primaria", resultado.get().getCurso());
    assertEquals("García López", resultado.get().getApellidos());
}

@Test
void actualizar_debeDevolverVacio_cuandoNoExiste() {
    when(repositorio.buscarPorId("999")).thenReturn(Optional.empty());

    Optional<AlumnoDTO> resultado = servicio.actualizar("999", new AlumnoDTO());

    assertTrue(resultado.isEmpty());
}

@Test
void actualizar_debeLanzarExcepcion_cuandoDniDuplicado() {
    AlumnoDTO dto = new AlumnoDTO();
    dto.setDni("87654321B");
```

```

        when(repositorio.existePorDniYIdDistinto("87654321B", "1")).thenReturn(true);

        assertThrows(NegocioException.class, () -> servicio.actualizar("1", dto));
    }

```

Pregunta: ¿Por qué el test de DNI duplicado verifica que se lanza la excepción antes de buscar el recurso?

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduces una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 10 - Escribir tests del controlador para PUT y PATCH

Añade tests al `AlumnoControllerTest`:

```

@Test
void actualizar_debeDevolver200_cuandoExiste() throws Exception {
    AlumnoDTO actualizado = new AlumnoDTO("1", "Ana", "García López", "12345678A",
        LocalDate.of(2010, 5, 12), "6º Primaria");
    when(service.actualizar(eq("1"), any(AlumnoDTO.class)))
        .thenReturn(Optional.of(actualizado));

    mockMvc.perform(put("/api/v1/alumnos/1")
        .contentType(MediaType.APPLICATION_JSON)
        .content("""
            {
                "nombre": "Ana",
                "apellidos": "García López",
                "dni": "12345678A",
                "fechaNacimiento": "2010-05-12",
                "curso": "6º Primaria"
            }
            """))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.curso").value("6º Primaria"));
}

@Test
void actualizar_debeDevolver404_cuandoNoExiste() throws Exception {
    when(service.actualizar(eq("999"), any(AlumnoDTO.class)))
        .thenReturn(Optional.empty());

    mockMvc.perform(put("/api/v1/alumnos/999")
        .contentType(MediaType.APPLICATION_JSON)
        .content("""
            { "nombre": "X", "apellidos": "Y", "dni": "00000000Z",
              "fechaNacimiento": "2010-01-01", "curso": "1º" }
            """))
        .andExpect(status().isNotFound());
}

```

```

}

@Test
void actualizarParcial_debeDevolver200() throws Exception {
    AlumnoDTO actualizado = new AlumnoDTO("1", "Ana", "García", "12345678A",
        LocalDate.of(2010, 5, 12), "6º Primaria B");
    when(service.actualizarParcial(eq("1"), anyMap()))
        .thenReturn(Optional.of(actualizado));

    mockMvc.perform(patch("/api/v1/alumnos/1")
        .contentType(MediaType.APPLICATION_JSON)
        .content("{\"curso\": \"6º Primaria B\"}"))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.curso").value("6º Primaria B"));
}

```

anyMap() es un matcher de Mockito que coincide con cualquier Map. eq("1") coincide exactamente con el string "1". Se usa cuando se mezclan matchers con valores concretos.

Pregunta: ¿Por qué en el test de PATCH se usa anyMap() en lugar de un Map concreto?

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduzcas una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta ./mvnw test (o mvnw.cmd test en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 11 - Errores comunes del ejercicio

Error	Causa	Solución
404 en un PUT de recurso existente	El ID de la URL no coincide	Verificar la URL usada en la petición
409 al actualizar el mismo recurso	Se valida el DNI contra el propio recurso	Usar existePorDniYIdDistinto(...)
500 en PATCH con fecha mal formateada	Falla LocalDate.parse(...)	Validar el formato y traducir el error a 400
PATCH no actualiza un campo	El campo no está en el Map	Verificar el nombre del campo
PATCH actualiza campos no enviados	Se usa get(...) sin containsKey(...)	Comprobar presencia antes de actualizar
415 Unsupported Media Type	Falta Content-Type	Añadir la cabecera application/json
No se detecta el DNI duplicado	Falta la comprobación específica	Añadir existePorDniYIdDistinto(...) al repositorio
ClassCastException en PATCH	El valor recibido no tiene el tipo esperado	Validar/converter el valor antes del cast

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduzcas una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 12 - Reto resuelto - Actualizar solo el estado de un expediente con PATCH

Reto: En el `ExpedienteService`, añadir un método `cambiarEstado` que reciba un ID y un nuevo estado, y actualice solo ese campo. Solución paso a paso: Paso 1: Añadir el método al servicio:

```
public Optional<ExpedienteDTO> cambiarEstado(String id, String nuevoEstado) {
    return repositorio.buscarPorId(id).map(expediente -> {
        expediente.setEstado(nuevoEstado);
        return repositorio.guardar(expediente);
    });
}
```

Paso 2: Añadir el endpoint al controlador:

```
@PatchMapping("/{id}/estado")
public ResponseEntity<ExpedienteDTO> cambiarEstado(
    @PathVariable String id,
    @RequestBody Map<String, String> cambios) {
    String nuevoEstado = cambios.get("estado");
    return service.cambiarEstado(id, nuevoEstado)
        .map(ResponseEntity::ok)
        .orElse(ResponseEntity.notFound().build());
}
```

Paso 3: Probar:

```
curl -i -X PATCH http://localhost:8080/api/v1/expedientes/1/estado \
-H "Content-Type: application/json" \
-d '{"estado": "RESUELTA"}'
```

Verás un 200 con el expediente actualizado. Solo ha cambiado el estado.

Pregunta: ¿Qué ventaja tiene tener un endpoint específico para cambiar el estado en lugar de un PATCH genérico?

Adaptación al snapshot final M3

Añade un PATCH específico en expediente para modificar el estado sin reenviar toda la representación.

Verificación

Comprueba **PATCH /api/v1/expedientes/{id}/estado** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Resultado esperado de la práctica

Al final del ejercicio, deberías tener:

- Un PUT que reemplaza completamente un recurso, con validación de DNI duplicado y 404 si no existe.
- Un PATCH que actualiza parcialmente un recurso, con validación de DNI duplicado.

- Un PATCH específico para cambiar el estado de un expediente.
- Tests del servicio y del controlador para PUT y PATCH.

Resumen técnico del ejercicio

El ejercicio ha demostrado:

- PUT: reemplazo completo. Idempotente. 200 o 404.
- PATCH: actualización parcial. No idempotente. 200 o 404.
- Validación de negocio: DNI duplicado en actualizaciones.
- existePorDniYIdDistinto: validación que excluye el propio recurso.
- LocalDate.parse: conversión de String a fecha.
- Tests: servicio con Mockito, controlador con MockMvc.
- Errores provocados: DNI duplicado, recurso no encontrado, fecha mal formateada.

Pregunta final

¿Qué evidencia mínima demuestra que la práctica 3.3 quedó integrada sin romper M2?

Respuesta razonada

La suite completa debe seguir verde y, además, una petición positiva y otra negativa representativas del punto deben producir el status y el cuerpo previstos. Esa combinación verifica regresión y comportamiento nuevo.

Práctica 3.4 - DELETE: eliminar recursos

Objetivo: aplicar de forma incremental el contrato de delete: eliminar recursos y conservar toda la funcionalidad heredada.

Paso 1 - Repasar el estado actual

Abre el AlumnoController y observa el método eliminar:

```
@DeleteMapping("/{id}")
public ResponseEntity<Void> eliminar(@PathVariable String id) {
    return service.eliminar(id)
        ? ResponseEntity.noContent().build()
        : ResponseEntity.notFound().build();
}
```

Y el método del AlumnoService:

```
public boolean eliminar(String id) {
    return repositorio.eliminar(id);
}
```

Y el método del AlumnoRepository:

```
public boolean eliminar(String id) {
    return almacen.remove(id) != null;
}
```

Arranca la aplicación y prueba:

```
# Eliminar un alumno existente
curl -i -X DELETE http://localhost:8080/api/v1/alumnos/2
```

```
# Intentar eliminar el mismo alumno otra vez
curl -i -X DELETE http://localhost:8080/api/v1/alumnos/2

# Consultar el alumno eliminado
curl -i http://localhost:8080/api/v1/alumnos/2
```

Pregunta: ¿Qué códigos de estado has visto en cada caso?

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduces una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 2 - Verificar el comportamiento esperado

Después de los tres comandos del paso anterior, deberías haber visto: 1. Primer DELETE: 204 No Content. El alumno 2 se eliminó. 2. Segundo DELETE: 404 Not Found. El alumno 2 ya no existía. 3. GET del alumno 2: 404 Not Found. El alumno 2 no existe. El estado final es consistente: el recurso no existe. Y el comportamiento es idempotente: la segunda llamada devuelve 404, pero el estado final es el mismo.

Pregunta: ¿Por qué el segundo DELETE devuelve 404 si la operación “eliminar” es idempotente?

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduces una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 3 - Añadir un segundo recurso para probar dependencias

Para probar el manejo de dependencias, necesitamos un segundo recurso que dependa del alumno. Vamos a crear un `DocumentoRepository` simple que gestione documentos asociados a alumnos. Crea la clase `DocumentoRepository` en el paquete `repository`:

```
package es.mecd.demo.miproyecto.repository;

import org.springframework.stereotype.Repository;

import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

@Repository
public class DocumentoRepository {
```

```

private final Map<String, String> documentos = new ConcurrentHashMap<>();

public DocumentoRepository() {
    documentos.put("doc1", "1");
    documentos.put("doc2", "1");
    documentos.put("doc3", "2");
}

public boolean existePorAlumnoId(String alumnoId) {
    return documentos.values().stream()
        .anyMatch(id -> id.equals(alumnoId));
}

public void eliminarPorAlumnoId(String alumnoId) {
    documentos.entrySet().removeIf(e -> e.getValue().equals(alumnoId));
}
}

```

documentos es un mapa donde la clave es el ID del documento y el valor es el ID del alumno al que pertenece. existePorAlumnoId comprueba si hay algún documento asociado a ese alumno. eliminarPorAlumnoId elimina todos los documentos del alumno. En el constructor, hemos añadido tres documentos: dos del alumno 1 y uno del alumno 2. Así podemos probar dependencias.

Pregunta: ¿Por qué el repositorio de documentos no conoce al repositorio de alumnos?

Adaptación al snapshot final M3

Crea DocumentoRepository dentro de la funcionalidad alumno. Cada repositorio gestiona su almacén; el servicio coordina.

Verificación

Comprueba **doc1 -> alumno 1, doc2 -> alumno 1, doc3 -> alumno 2** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 4 - Añadir validación de dependencias en el servicio

Vamos a modificar el AlumnoService para que compruebe si el alumno tiene documentos antes de eliminarlo. Primero, inyecta el DocumentoRepository:

```

@Service
public class AlumnoService {

    private final AlumnoRepository repositorio;
    private final DocumentoRepository documentoRepository;

    public AlumnoService(AlumnoRepository repositorio,
        DocumentoRepository documentoRepository) {
        this.repositorio = repositorio;
        this.documentoRepository = documentoRepository;
    }

    // ... resto de métodos

    public boolean eliminar(String id) {
        if (documentoRepository.existePorAlumnoId(id)) {
            throw new NegocioException(

```

```

        "No se puede eliminar el alumno " + id
        + ": tiene documentos asociados");
    }
    return repositorio.eliminar(id);
}
}

```

La inyección por constructor ahora recibe dos repositorios: el de alumnos y el de documentos. `documentoRepository.existePorAlumnoId(id)` comprueba si hay documentos asociados. `throw new NegocioException(...)` lanza la excepción si hay dependencias.

Pregunta: ¿Por qué la comprobación de dependencias va en el servicio y no en el repositorio?

Adaptación al snapshot final M3

Antes de eliminar, si existen documentos asociados lanza `NegocioException` salvo en la operación de cascada explícita.

Verificación

Comprueba **No se puede eliminar ... tiene documentos asociados** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 5 - Probar la eliminación con dependencias

Reinicia la aplicación y prueba a eliminar un alumno que tiene documentos:

```
curl -i -X DELETE http://localhost:8080/api/v1/alumnos/1
```

El alumno 1 tiene dos documentos asociados (doc1 y doc2). Verás un 409 Conflict con el mensaje:

```

{
  "status": 409,
  "error": "Conflict",
  "message": "No se puede eliminar el alumno 1: tiene documentos asociados"
}

```

El servicio detecta las dependencias y lanza `NegocioException`; el manejador la traduce a 409.

Pregunta: ¿Por qué es 409 y no 400? ¿Qué diferencia hay entre un conflicto y una petición mal formada?

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduces una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 6 - Probar la eliminación sin dependencias

Ahora prueba a eliminar un alumno que no tiene documentos. El alumno 2 tiene un documento (doc3), así que no podemos usarlo. Vamos a crear un alumno nuevo sin documentos:

```
# Crear alumno nuevo
curl -i -X POST http://localhost:8080/api/v1/alumnos \
  -H "Content-Type: application/json" \
  -d '{"nombre":"Pedro","apellidos":"Sánchez","dni":"77777777J","fechaNacimiento":"2010-07-15","cu
```

Ahora elimina ese alumno:

```
curl -i -X DELETE http://localhost:8080/api/v1/alumnos/3
```

Verás un 204 No Content. El alumno 3 no tenía documentos, así que se ha eliminado sin problema.

Pregunta: ¿Qué pasaría si eliminaras el alumno 2? ¿Tiene documentos?

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduces una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 7 - Probar la eliminación en cascada

Vamos a implementar la eliminación en cascada para que, si se quiere eliminar un alumno con documentos, se eliminen también los documentos. Modifica el servicio:

```
public boolean eliminarConCascada(String id) {
    documentoRepository.eliminarPorAlumnoId(id);
    return repositorio.eliminar(id);
}
```

`documentoRepository.eliminarPorAlumnoId(id)` elimina todos los documentos del alumno. `repositorio.eliminar(id)` elimina el alumno. Esta operación debería ser transaccional: si falla la eliminación del alumno, no deberían haberse eliminado los documentos. En el Módulo 4, cuando usemos JPA, veremos cómo gestionar transacciones con `@Transactional`. Por ahora, la implementación es secuencial. Añade un endpoint para la eliminación en cascada:

```
@DeleteMapping("/{id}/cascada")
public ResponseEntity<Void> eliminarConCascada(@PathVariable String id) {
    return service.eliminarConCascada(id)
        ? ResponseEntity.noContent().build()
        : ResponseEntity.notFound().build();
}
```

Prueba:

```
curl -i -X DELETE http://localhost:8080/api/v1/alumnos/1/cascada
```

Verás un 204 No Content. El alumno 1 y sus documentos se han eliminado.

Pregunta: ¿Qué diferencia hay entre eliminar con restricción y eliminar en cascada?

Adaptación al snapshot final M3

La variante en cascada elimina documentos asociados y después aplica la eliminación del alumno.

Verificación

Comprueba **DELETE /api/v1/alumnos/{id}/cascada** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 8 - Escribir tests del servicio para DELETE

Añade tests al `AlumnoServiceTest`:

```
@Test
void eliminar_debeDevolverTrue_cuandoExiste() {
    when(documentoRepository.existePorAlumnoId("1")).thenReturn(false);
    when(repositorio.eliminar("1")).thenReturn(true);

    boolean resultado = servicio.eliminar("1");

    assertTrue(resultado);
    verify(repositorio).eliminar("1");
}

@Test
void eliminar_debeDevolverFalse_cuandoNoExiste() {
    when(documentoRepository.existePorAlumnoId("999")).thenReturn(false);
    when(repositorio.eliminar("999")).thenReturn(false);

    boolean resultado = servicio.eliminar("999");

    assertFalse(resultado);
}

@Test
void eliminar_debeLanzarExcepcion_cuandoTieneDependencias() {
    when(documentoRepository.existePorAlumnoId("1")).thenReturn(true);

    assertThrows(NegocioException.class, () -> servicio.eliminar("1"));
    verify(repositorio, never()).eliminar("1");
}
```

`verify(repositorio, never()).eliminar("1")` verifica que no se ha llamado a eliminar en el repositorio. Eso es importante: si hay

dependencias, no se debe eliminar nada.

Pregunta: ¿Por qué el test de dependencias verifica que no se ha llamado a eliminar?

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduzcas una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 9 - Escribir tests del controlador para DELETE

Añade tests al AlumnoControllerTest:

```
@Test
void eliminar_debeDevolver204_cuandoExiste() throws Exception {
    when(service.eliminar("1")).thenReturn(true);

    mockMvc.perform(delete("/api/v1/alumnos/1"))
        .andExpect(status().isNoContent());
}

@Test
void eliminar_debeDevolver404_cuandoNoExiste() throws Exception {
    when(service.eliminar("999")).thenReturn(false);

    mockMvc.perform(delete("/api/v1/alumnos/999"))
        .andExpect(status().isNotFound());
}

@Test
void eliminar_debeDevolver409_cuandoTieneDependencias() throws Exception {
    when(service.eliminar("1"))
        .thenReturn(new NegocioException("Tiene documentos asociados"));

    mockMvc.perform(delete("/api/v1/alumnos/1"))
        .andExpect(status().isConflict())
        .andExpect(jsonPath("$.status").value(409));
}
```

delete("/api/v1/alumnos/1") simula una petición DELETE. status().isNoContent() verifica que el código es 204. status().isNotFound() verifica que el código es 404.

Pregunta: ¿Por qué el test del controlador no necesita un repositorio real?

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduces una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta ./mvnw test (o mvnw.cmd test en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 10 - Probar la idempotencia de DELETE

Vamos a probar la idempotencia de DELETE con curl. Primero, crea un alumno nuevo:

```
curl -X POST http://localhost:8080/api/v1/alumnos \
  -H "Content-Type: application/json" \
  -d '{"nombre":"Test","apellidos":"Idempotencia","dni":"99999999K","fechaNacimiento":"2010-01-01"}
```

Ahora elimínalo dos veces y observa los códigos:

```
# Primera eliminación
curl -i -X DELETE http://localhost:8080/api/v1/alumnos/4
```

Segunda eliminación

```
curl -i -X DELETE http://localhost:8080/api/v1/alumnos/4
```

La primera petición elimina el recurso. La segunda no debe recrear ningún efecto: el estado final sigue siendo “recurso ausente”. Eso es la propiedad de idempotencia que queremos observar.

Primera: 204 No Content.

- Segunda: 404 Not Found. El estado final es el mismo: el recurso no existe. Eso es la idempotencia. El código de la segunda llamada es distinto (404 en lugar de 204), pero el efecto es el mismo. **Pregunta: ¿Por qué se dice que DELETE es idempotente aunque la segunda llamada devuelva 404?**

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduces una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 11 - Errores comunes del ejercicio

Error	Causa	Solución
404 en DELETE de recurso existente	El ID no coincide	Verificar la URL
409 en DELETE sin dependencias	La comprobación de dependencias es incorrecta	Revisar <code>existePorAlumnoId(...)</code>
500 en DELETE	La excepción conocida no está traducida	Añadir el manejo correspondiente
El recurso no se elimina	El repositorio no ejecuta la eliminación	Verificar <code>remove(...)</code>
415 Unsupported Media Type	Se envía un cuerpo innecesario	No enviar cuerpo en DELETE
405 Method Not Allowed	Se usa GET en lugar de DELETE	Usar el método HTTP correcto
El test falla por <code>never()</code>	Se llegó a llamar a <code>eliminar(...)</code>	Revisar la regla de dependencias
La cascada no elimina documentos	Falta <code>eliminarPorAlumnoId(...)</code>	Añadir la operación al repositorio de documentos

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduces una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 12 - Reto resuelto - Soft delete de alumnos

Reto: Implementar soft delete para los alumnos. En lugar de eliminar físicamente, marcar el alumno como eliminado. Las consultas deben excluir los alumnos eliminados. Solución paso a paso: Paso 1: Añadir el campo eliminado al AlumnoDTO:

```
@JsonIgnore
private Boolean eliminado = false;

public Boolean getEliminado() { return eliminado; }
public void setEliminado(Boolean eliminado) { this.eliminado = eliminado; }
@JsonIgnore
```

excluye el campo de la serialización JSON. El cliente no ve el campo eliminado.

Paso 2: Modificar el método eliminar del repositorio para hacer soft delete:

```
public boolean eliminar(String id) {
    return almacen.computeIfPresent(id, (k, v) -> {
        v.setEliminado(true);
        return v;
    }) != null;
}
```

computeIfPresent ejecuta la función solo si la clave existe. Marca el recurso como eliminado y lo devuelve. Paso 3: Modificar las consultas para excluir los eliminados:

```
public Optional<AlumnoDTO> buscarPorId(String id) {
    return Optional.ofNullable(almacen.get(id))
        .filter(a -> !a.getEliminado());
}

public List<AlumnoDTO> listarTodos() {
    return almacen.values().stream()
        .filter(a -> !a.getEliminado())
        .toList();
}
```

filter(a -> !a.getEliminado()) excluye los alumnos marcados como eliminados. Paso 4: Probar:

Crear alumno

```
curl -X POST http://localhost:8080/api/v1/alumnos \
  -H "Content-Type: application/json" \
  -d '{"nombre":"Soft","apellidos":"Delete","dni":"10101010L","fechaNacimiento":"2010-01-01","curs
```

Eliminar (soft delete)

```
curl -i -X DELETE http://localhost:8080/api/v1/alumnos/5
```

Consultar (debe devolver 404)

```
curl -i http://localhost:8080/api/v1/alumnos/5
```

Listar (no debe aparecer)

```
curl http://localhost:8080/api/v1/alumnos
```

Verás que el alumno 5 no aparece en las consultas, pero sigue en el almacén (marcado como eliminado).

Pregunta: ¿Qué ventaja tiene soft delete frente a hard delete en este caso?

Adaptación al snapshot final M3

Añade eliminado al modelo interno y convierte DELETE ordinario sin dependencias en borrado lógico; las consultas excluyen eliminados.

Verificación

Comprueba **activo en la respuesta se deriva de !eliminado** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Resultado esperado de la práctica

Al final del ejercicio, deberías tener:

- Un DELETE que devuelve 204 o 404.
- Validación de dependencias: 409 si el alumno tiene documentos.
- (Opcional) Eliminación en cascada.
- (Opcional) Soft delete.
- Tests del servicio y del controlador para DELETE.

Resumen técnico del ejercicio

El ejercicio ha demostrado:

- DELETE: eliminar un recurso. 204 o 404.
- Idempotencia: la segunda llamada devuelve 404, pero el estado final es el mismo.
- Dependencias: restricción (409) o cascada.
- Soft delete: marcar como eliminado en lugar de borrar.
- Tests: servicio con Mockito, controlador con MockMvc.
- Errores provocados: recurso no encontrado, dependencias.

Pregunta final

¿Qué evidencia mínima demuestra que la práctica 3.4 quedó integrada sin romper M2?

Respuesta razonada

La suite completa debe seguir verde y, además, una petición positiva y otra negativa representativas del punto deben producir el status y el cuerpo previstos. Esa combinación verifica regresión y comportamiento nuevo.

Práctica 3.5 - DTOs avanzados y validaciones

Objetivo: aplicar de forma incremental el contrato de dtos avanzados y validaciones y conservar toda la funcionalidad heredada.

Paso 1 - Añadir la dependencia de validación

Abre el `pom.xml` y verifica que tienes la dependencia `spring-boot-starter-validation`:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

Si no la tienes, añádela. Spring Boot la incluye dentro de spring-boot-starter-web en algunas versiones, pero es mejor declararla explícitamente.

Pregunta: ¿Por qué conviene declarar explícitamente la dependencia de validación?

Adaptación al snapshot final M3

Añade spring-boot-starter-validation sin versión explícita porque la gestiona el parent de Spring Boot.

Verificación

Comprueba **jakarta.validation** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 2 - Crear el DTO de entrada AlumnoRequestDTO

Crea la clase AlumnoRequestDTO en el paquete dto:

```
package es.mecd.demo.miproyecto.dto;

import com.fasterxml.jackson.annotation.JsonFormat;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;

import java.time.LocalDate;

public class AlumnoRequestDTO {

    @NotBlank(message = "El nombre es obligatorio")
    @Size(max = 100, message = "El nombre no puede superar 100 caracteres")
    private String nombre;

    @NotBlank(message = "Los apellidos son obligatorios")
    @Size(max = 150, message = "Los apellidos no pueden superar 150 caracteres")
    private String apellidos;

    @NotBlank(message = "El DNI es obligatorio")
    @Size(min = 9, max = 9, message = "El DNI debe tener 9 caracteres")
    private String dni;

    @NotNull(message = "La fecha de nacimiento es obligatoria")
    @JsonFormat(pattern = "yyyy-MM-dd")
    private LocalDate fechaNacimiento;

    @NotBlank(message = "El curso es obligatorio")
    private String curso;

    public AlumnoRequestDTO() {
    }

    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }

    public String getApellidos() { return apellidos; }
```

```

    public void setApellidos(String apellidos) { this.apellidos = apellidos; }

    public String getDni() { return dni; }
    public void setDni(String dni) { this.dni = dni; }

    public LocalDate getFechaNacimiento() { return fechaNacimiento; }
    public void setFechaNacimiento(LocalDate fechaNacimiento) { this.fechaNacimiento = fechaNacimiento; }

    public String getCurso() { return curso; }
    public void setCurso(String curso) { this.curso = curso; }
}

```

- @NotBlank valida que el campo no sea nulo, vacío ni sólo espacios.
- @Size(min = 9, max = 9) valida que el DNI tenga exactamente nueve caracteres.
- @NotNull valida que la fecha no sea nula.
- @JsonFormat indica el formato de la fecha al deserializar.

El DTO de entrada no tiene id ni activo porque el cliente no puede establecerlos.

Pregunta: ¿Qué diferencia hay entre @NotNull y @NotBlank?

Adaptación al snapshot final M3

El request contiene sólo campos editables y constraints declarativas: nombre/apellidos/curso no vacíos, DNI de 9 caracteres, fecha obligatoria y pasada.

Verificación

Comprueba **AlumnoRequestDTO** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 3 - Crear el DTO de salida AlumnoResponseDTO

Crea la clase AlumnoResponseDTO en el paquete dto:

```

package es.mecd.demo.miproyecto.dto;

import com.fasterxml.jackson.annotation.JsonFormat;
import com.fasterxml.jackson.annotation.JsonInclude;
import com.fasterxml.jackson.annotation.JsonProperty;

import java.time.LocalDate;

@JsonInclude(JsonInclude.Include.NON_NULL)
public class AlumnoResponseDTO {

    @JsonProperty("id")
    private String identificador;

    private String nombre;
    private String apellidos;
    private String dni;

    @JsonFormat(pattern = "yyyy-MM-dd")
    private LocalDate fechaNacimiento;

    private String curso;
}

```

```

private Boolean activo;

public AlumnoResponseDTO() {
}

public AlumnoResponseDTO(String identificador, String nombre, String apellidos,
                          String dni, LocalDate fechaNacimiento, String curso) {
    this.identificador = identificador;
    this.nombre = nombre;
    this.apellidos = apellidos;
    this.dni = dni;
    this.fechaNacimiento = fechaNacimiento;
    this.curso = curso;
    this.activo = true;
}

public String getIdentificador() { return identificador; }
public void setIdentificador(String identificador) { this.identificador = identificador; }

public String getNombre() { return nombre; }
public void setNombre(String nombre) { this.nombre = nombre; }

public String getApellidos() { return apellidos; }
public void setApellidos(String apellidos) { this.apellidos = apellidos; }

public String getDni() { return dni; }
public void setDni(String dni) { this.dni = dni; }

public LocalDate getFechaNacimiento() { return fechaNacimiento; }
public void setFechaNacimiento(LocalDate fechaNacimiento) { this.fechaNacimiento = fechaNacimiento; }

public String getCurso() { return curso; }
public void setCurso(String curso) { this.curso = curso; }

public Boolean getActivo() { return activo; }
public void setActivo(Boolean activo) { this.activo = activo; }
}

```

El DTO de salida tiene id y activo, que el cliente no puede establecer. No tiene validaciones porque los datos ya están validados.

Pregunta: ¿Por qué el DTO de salida tiene un campo activo que el DTO de entrada no tiene?

Adaptación al snapshot final M3

El response contiene id y estado derivado, pero no constraints de entrada.

Verificación

Comprueba **AlumnoResponseDTO** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 4 - Modificar el controlador para usar los nuevos DTOs

Vamos a modificar el AlumnoController para que use AlumnoRequestDTO en el POST y AlumnoResponseDTO en las respuestas. Primero, añade un método de transformación en el servicio:

```
private AlumnoResponseDTO toResponseDTO(AlumnoDTO dto) {
    AlumnoResponseDTO response = new AlumnoResponseDTO();
    response.setIdentificador(dto.getIdentificador());
    response.setNombre(dto.getNombre());
    response.setApellidos(dto.getApellidos());
    response.setDni(dto.getDni());
    response.setFechaNacimiento(dto.getFechaNacimiento());
    response.setCurso(dto.getCurso());
    response.setActivo(true);
    return response;
}

private AlumnoDTO toDTO(AlumnoRequestDTO request) {
    AlumnoDTO dto = new AlumnoDTO();
    dto.setNombre(request.getNombre());
    dto.setApellidos(request.getApellidos());
    dto.setDni(request.getDni());
    dto.setFechaNacimiento(request.getFechaNacimiento());
    dto.setCurso(request.getCurso());
    return dto;
}
```

toResponseDTO convierte el DTO interno a DTO de salida. toDTO convierte el DTO de entrada a DTO interno. Ahora modifica el método crear del servicio para que reciba AlumnoRequestDTO:

```
public AlumnoResponseDTO crear(AlumnoRequestDTO request) {
    AlumnoDTO dto = toDTO(request);
    if (repositorio.existePorDni(dto.getDni())) {
        throw new NegocioException("Ya existe un alumno con el DNI " + dto.getDni());
    }
    if (dto.getIdentificador() == null || dto.getIdentificador().isBlank()) {
        dto.setIdentificador(String.valueOf(repositorio.contar() + 1));
    }
    return toResponseDTO(repositorio.guardar(dto));
}
```

Y modifica el controlador:

```
@PostMapping
public ResponseEntity<AlumnoResponseDTO> crear(
    @Valid @RequestBody AlumnoRequestDTO request) {
    AlumnoResponseDTO creado = service.crear(request);
    URI location = ServletUriComponentsBuilder
        .fromCurrentRequest()
        .path("/{id}")
        .buildAndExpand(creado.getIdentificador())
        .toUri();
    return ResponseEntity.created(location).body(creado);
}
```

@Valid activa la validación del DTO de entrada. Si hay errores, Spring MVC lanza MethodArgumentNotValidException antes de ejecutar

el método.

Pregunta: ¿Qué pasa si el cliente envía un POST con un DTO de entrada que tiene un campo id? ¿Se tiene en cuenta?

Adaptación al snapshot final M3

Controlador recibe @Valid @RequestBody AlumnoRequestDTO; el servicio transforma al modelo interno y devuelve response.

Verificación

Comprueba @Valid y ejecuta ./mvnw test (o mvnw.cmd test en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 5 - Añadir el manejador de errores de validación

Añade el manejador al controlador:

```
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<Map<String, Object>> handleValidation(
    MethodArgumentNotValidException ex) {
    List<Map<String, String>> errores = ex.getBindingResult()
        .getFieldErrors()
        .stream()
        .map(e -> Map.of(
            "field", e.getField(),
            "message", e.getDefaultMessage()))
        .toList();

    Map<String, Object> respuesta = Map.of(
        "timestamp", java.time.Instant.now().toString(),
        "status", 400,
        "error", "Bad Request",
        "message", "Errores de validación",
        "errors", errores
    );
    return ResponseEntity.badRequest().body(respuesta);
}
```

getFieldErrors() devuelve la lista de errores por campo. e.getField() devuelve el nombre del campo. e.getDefaultMessage() devuelve el mensaje de validación. Map.of(...) construye la respuesta con los errores.

Pregunta: ¿Por qué se incluye el campo field en cada error? ¿Qué utilidad tiene para el cliente?

Adaptación al snapshot final M3

Centraliza MethodArgumentNotValidException en @RestControllerAdvice y devuelve una lista {field,message}.

Verificación

Comprueba **400 con errors** y ejecuta ./mvnw test (o mvnw.cmd test en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 6 - Arrancar y probar el POST con datos válidos

Reinicia la aplicación y prueba:

```
curl -i -X POST http://localhost:8080/api/v1/alumnos \
  -H "Content-Type: application/json" \
  -d '{"nombre":"María","apellidos":"López","dni":"11111111C","fechaNacimiento":"2011-03-20","curso":1}'
```

Verás un 201 con el recurso creado. El DTO de salida incluye id y activo.

Pregunta: ¿Qué campos del DTO de salida no estaban en el DTO de entrada?

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduces una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 7 - Probar el POST con datos inválidos

Ahora prueba con datos inválidos:

```
curl -i -X POST http://localhost:8080/api/v1/alumnos \
  -H "Content-Type: application/json" \
  -d '{"nombre":"","apellidos":"López","dni":"123","fechaNacimiento":"2011-03-20","curso":4}'
```

Verás un 400 con la lista de errores:

```
{
  "timestamp": "...",
  "status": 400,
  "error": "Bad Request",
  "message": "Errores de validación",
  "errors": [
    {"field": "nombre", "message": "El nombre es obligatorio"},
    {"field": "dni", "message": "El DNI debe tener 9 caracteres"}
  ]
}
```

El cliente recibe dos errores: el nombre está vacío y el DNI tiene 3 caracteres en lugar de 9.

Pregunta: ¿Qué pasaría si el cliente enviara solo un error? ¿La respuesta sería igual?

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduces una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 8 - Probar el POST sin fecha de nacimiento

Prueba con un POST sin fecha de nacimiento:

```
curl -i -X POST http://localhost:8080/api/v1/alumnos \
-H "Content-Type: application/json" \
-d '{"nombre":"Pedro","apellidos":"Sánchez","dni":"22222222D","curso":"4º"}'
```

Verás un 400 con el error:

```
{
  "errors": [
    { "field": "fechaNacimiento", "message": "La fecha de nacimiento es obligatoria" }
  ]
}
```

Pregunta: ¿Qué diferencia hay entre @NotNull y @NotBlank en este caso?

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduces una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 9 - Probar el POST con DNI duplicado

Prueba a crear un alumno con un DNI que ya existe:

```
curl -i -X POST http://localhost:8080/api/v1/alumnos \
-H "Content-Type: application/json" \
-d '{"nombre":"Otro","apellidos":"Alumno","dni":"12345678A","fechaNacimiento":"2010-01-01","curso":"4º"}'
```

Verás un 409 Conflict. Bean Validation ha pasado porque el DNI tiene nueve caracteres, pero la regla de negocio falla porque el DNI ya está registrado.

Pregunta: ¿Qué diferencia hay entre el 400 de validación de formato y el 409 de validación de negocio?

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduces una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 10 - Escribir tests para validaciones

Añade tests al `AlumnoControllerTest`:

```

@Test
void crear_debeDevolver400_cuandoNombreVacio() throws Exception {
    mockMvc.perform(post("/api/v1/alumnos")
        .contentType(MediaType.APPLICATION_JSON)
        .content("""
            {"nombre":"","apellidos":"López",
            "dni":"12345678A", "fechaNacimiento":"2011-03-20",
            "curso":"4º"}
            """))
        .andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.errors[?(@.field=='nombre')].message")
            .value("El nombre es obligatorio"));
}

@Test
void crear_debeDevolver400_cuandoDniCorto() throws Exception {
    mockMvc.perform(post("/api/v1/alumnos")
        .contentType(MediaType.APPLICATION_JSON)
        .content("""
            {"nombre":"María", "apellidos":"López",
            "dni":"123", "fechaNacimiento":"2011-03-20",
            "curso":"4º"}
            """))
        .andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.errors[?(@.field=='dni')].message")
            .value("El DNI debe tener 9 caracteres"));
}

@Test
void crear_debeDevolver400_cuandoFaltanCampos() throws Exception {
    mockMvc.perform(post("/api/v1/alumnos")
        .contentType(MediaType.APPLICATION_JSON)
        .content("""
            {"nombre":"María", "apellidos":"López"}
            """))
        .andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.errors.length()").value(3));
}

```

jsonPath("\$.errors.length()").value(3) verifica que hay 3 errores: DNI, fecha de nacimiento y curso.

Pregunta: ¿Por qué el test de campos faltantes espera 3 errores y no 2?

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduces una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 11 - Errores comunes del ejercicio

Error	Causa	Solución
400 Bad Request sin detalles útiles	Falta el manejador de validación	Traducir MethodArgumentNotValidException
500 Internal Server Error durante validación	La excepción de validación no se gestiona	Añadir el manejador centralizado
La validación no se ejecuta	Falta @Valid	Añadir @Valid al parámetro de entrada
Un campo no se valida	Falta la constraint	Añadir @NotBlank, @Size, etc.
El mensaje es genérico	No se especificó message	Definir un mensaje de validación claro
415 Unsupported Media Type	Falta Content-Type	Añadir application/json
409 Conflict en lugar de 400	Es una regla de negocio, no de formato	Revisar si el caso realmente es conflicto
El test de validación falla	El JSON del test está mal formado	Revisar el contenido enviado

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduces una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 12 - Reto resuelto - DTO anidado para Expediente

Reto: Crear un `ExpedienteRequestDTO` con un `SolicitanteDTO` anidado, aplicando validaciones a ambos. Solución paso a paso: Paso 1: Crear `SolicitanteDTO`:

```
package es.mecd.demo.miproyecto.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

public class SolicitanteDTO {

    @NotBlank(message = "El nombre del solicitante es obligatorio")
    private String nombre;

    @NotBlank(message = "Los apellidos del solicitante son obligatorios")
    private String apellidos;

    @NotBlank(message = "El DNI del solicitante es obligatorio")
    @Size(min = 9, max = 9, message = "El DNI debe tener 9 caracteres")
    private String dni;

    public SolicitanteDTO() {
    }

    // getters y setters
}
```

Paso 2: Crear `ExpedienteRequestDTO`:

```

package es.mecd.demo.miproyecto.dto;

import jakarta.validation.Valid;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.PositiveOrZero;

import java.time.LocalDate;

public class ExpedienteRequestDTO {

    @NotNull(message = "El solicitante es obligatorio")
    @Valid
    private SolicitanteDTO solicitante;

    @NotBlank(message = "El tipo es obligatorio")
    private String tipo;

    @NotNull(message = "La fecha de solicitud es obligatoria")
    private LocalDate fechaSolicitud;

    @PositiveOrZero(message = "El importe debe ser positivo o cero")
    private Double importe;

    public ExpedienteRequestDTO() {
    }

    // getters y setters
}

```

@Valid en el campo solicitante activa la validación del DTO anidado. Sin esa anotación, las constraints de SolicitanteDTO no se evaluarían.

ejecutarían. Paso 3: Probar con un POST que tenga un solicitante inválido:

```

curl -i -X POST http://localhost:8080/api/v1/expedientes \
-H "Content-Type: application/json" \
-d '{
  "solicitante":{"nombre":"","apellidos":"García","dni":"123"},
  "tipo":"BECA",
  "fechaSolicitud":"2025-01-15",
  "importe":1500.0
}'

```

Verás un 400 con los errores del solicitante anidado:

```

{
  "errors": [
    {"field": "solicitante.nombre", "message": "El nombre del solicitante es obligatorio"},
    {"field": "solicitante.dni", "message": "El DNI debe tener 9 caracteres"}
  ]
}

```

El campo se reporta como solicitante.nombre, indicando la ruta dentro del objeto anidado.

Pregunta: ¿Por qué es necesario @Valid en el campo anidado? ¿Qué pasaría si no estuviera?

Adaptación al snapshot final M3

Crea ExpedienteRequestDTO con @Valid SolicitanteDTO y constraints en ambos niveles.

Verificación

Comprueba @Valid anidado y ejecuta ./mvnw test (o mvnw.cmd test en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Resultado esperado de la práctica

Al final del ejercicio, deberías tener:

- Un AlumnoRequestDTO con validaciones.
- Un AlumnoResponseDTO sin validaciones.
- Un AlumnoController que usa ambos DTOs y @Valid.
- Un @ExceptionHandler que captura MethodArgumentNotValidException y devuelve 400 con los errores.
- Tests que verifican las validaciones.
- (Opcional) Un ExpedienteRequestDTO con SolicitanteDTO anidado.

Resumen técnico del ejercicio

El ejercicio ha demostrado:

- DTOs de entrada y salida: separados por seguridad, validación y claridad.
- DTOs anidados: con @Valid para validar los sub-DTOs.
- Bean Validation: @NotBlank, @Size, @NotNull, @PositiveOrZero.
- @Valid: activa la validación en el controlador.
- MethodArgumentNotValidException: captura los errores y los devuelve en un 400.
- Tests: verificar 400 y los mensajes de error.

Pregunta final

¿Qué evidencia mínima demuestra que la práctica 3.5 quedó integrada sin romper M2?

Respuesta razonada

La suite completa debe seguir verde y, además, una petición positiva y otra negativa representativas del punto deben producir el status y el cuerpo previstos. Esa combinación verifica regresión y comportamiento nuevo.

Práctica 3.6 - Parámetros, cabeceras y Postman

Objetivo: aplicar de forma incremental el contrato de parámetros, cabeceras y postman y conservar toda la funcionalidad heredada.

Paso 1 - Añadir un endpoint que lea cabeceras

Vamos a añadir un endpoint que devuelva información sobre la petición: el User-Agent, el Accept-Language y el Host. Añade el método al AlumnoController:

```
@GetMapping("/info-peticion")
public Map<String, String> infoPeticion(
    @RequestHeader(value = "User-Agent", required = false) String userAgent,
    @RequestHeader(value = "Accept-Language", required = false) String idioma,
```

```

    @RequestHeader(value = "Host", required = false) String host) {
    return Map.of(
        "userAgent", userAgent != null ? userAgent : "desconocido",
        "idioma", idioma != null ? idioma : "desconocido",
        "host", host != null ? host : "desconocido"
    );
}

```

@RequestHeader(value = "User-Agent", required = false) captura la cabecera User-Agent. Si no está, devuelve null. @RequestHeader(value = "Accept-Language", required = false) captura la cabecera Accept-Language. @RequestHeader(value = "Host", required = false) captura la cabecera Host.

El método devuelve un Map con los valores. Si alguno es null, se sustituye por "desconocido". Reinicia y prueba:

```
curl http://localhost:8080/api/v1/alumnos/info-peticion
```

Verás algo como:

```

{
  "userAgent": "curl/8.4.0",
  "idioma": "desconocido",
  "host": "localhost:8080"
}

```

Pregunta: ¿Qué pasa si envías la cabecera Accept-Language en el curl? ¿Cómo se envía una cabecera con curl?

Adaptación al snapshot final M3

Expón un endpoint pedagógico que lea User-Agent, Accept-Language y una cabecera propia opcional.

Verificación

Comprueba @RequestHeader y ejecuta ./mvnw test (o mvnw.cmd test en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 2 - Probar con cabeceras personalizadas

Vamos a enviar cabeceras personalizadas con curl. El flag -H permite añadir cabeceras:

```

curl http://localhost:8080/api/v1/alumnos/info-peticion \
  -H "User-Agent: MiCliente/1.0" \
  -H "Accept-Language: es-ES"

```

Verás algo como:

```

{
  "userAgent": "MiCliente/1.0",
  "idioma": "es-ES",
  "host": "localhost:8080"
}

```

Pregunta: ¿Qué diferencia hay entre el User-Agent que envía curl por defecto y el que enviamos con -H?

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduzcas una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 3 - Añadir un endpoint que lea cookies

Vamos a añadir un endpoint que lea una cookie llamada preferencias:

```
@GetMapping("/info-cookies")
public Map<String, String> infoCookies(
    @CookieValue(value = "preferencias", required = false) String preferencias) {
    return Map.of(
        "preferencias", preferencias != null ? preferencias : "sin preferencias"
    );
}
```

`@CookieValue(value = "preferencias", required = false)` captura la cookie preferencias. Si no existe, devuelve null.

Reinicia y prueba sin cookies:

```
curl http://localhost:8080/api/v1/alumnos/info-cookies
```

Verás:

```
{"preferencias": "sin preferencias"}
```

Ahora prueba con una cookie:

```
curl http://localhost:8080/api/v1/alumnos/info-cookies \
  -b "preferencias=es-ES"
```

Verás:

```
{"preferencias": "es-ES"}
```

La opción `-b "preferencias=es-ES"` envía la cookie preferencias con valor es-ES.

Pregunta: ¿Qué diferencia hay entre una cookie y una cabecera?

Adaptación al snapshot final M3

Lee una cookie de sesión opcional sin convertirla en requisito funcional.

Verificación

Comprueba `@CookieValue` y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 4 - Añadir un endpoint que acepte varios valores en un parámetro

Vamos a añadir un endpoint que acepte varios cursos como lista:

```
@GetMapping("/por-cursos")
public List<AlumnoDTO> porCursos(@RequestParam List<String> cursos) {
    return service.listar().stream()
        .filter(a -> cursos.contains(a.getCurso()))
        .toList();
}
```

@RequestParam List<String> cursos captura varios valores del mismo parámetro. Si el cliente envía ?cursos=5º&cursos=6º, Spring construye una lista con ambos valores.

tendrá dos elementos. Reinicia y prueba:

```
curl "http://localhost:8080/api/v1/alumnos/por-cursos?cursos=5º Primaria&cursos=6º Primaria"
```

Verás los alumnos de esos dos cursos.

Pregunta: ¿Qué pasa si el cliente no envía el parámetro cursos? ¿La lista estará vacía o será null?

Adaptación al snapshot final M3

Acepta el mismo query parameter varias veces mediante List<String>.

Verificación

Comprueba **curso=4º&curso=5º** y ejecuta ./mvnw test (o mvnw.cmd test en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 5 - Instalar Postman

Descarga Postman desde postman.com/downloads/. Instálalo con las opciones por defecto. Al abrirlo, verás la interfaz principal con:

- Barra lateral izquierda: colecciones, historial, entornos.
- Área central: la petición actual.
- Barra superior: método, URL, botón Send. **Pregunta: ¿Qué ventaja tiene Postman frente a curl?**

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduzcas una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta ./mvnw test (o mvnw.cmd test en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 6 - Crear una colección en Postman

Vamos a crear una colección para el proyecto. En Postman: 1. Haz clic en New > Collection. 2. Nombra la colección "Mi Proyecto". 3. Añade una descripción: "Colección de pruebas para el proyecto del curso". 4. Crea una carpeta dentro de la colección llamada "Alumnos". 5. Dentro de la carpeta, crea una petición.

Pregunta: ¿Por qué es útil organizar las peticiones en colecciones y carpetas?

Adaptación al snapshot final M3

Agrupar GET/POST/PUT/PATCH/DELETE en una colección exportable dentro de M3/postman.

Verificación

Comprueba **M3.postman_collection.json** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 7 - Crear un entorno en Postman

Vamos a crear un entorno para no repetir la URL base. En Postman: 1. Haz clic en Environments > Create Environment. 2. Nombra el entorno "Local". 3. Añade una variable baseUrl con valor `http://localhost:8080`. 4. Guarda el entorno. 5. Selecciona el entorno "Local" en el desplegable superior derecho. Ahora, en las peticiones, puedes usar `{{baseUrl}}` en lugar de `http://localhost:8080`. Si mañana cambia el puerto, solo hay que cambiar la variable.

Pregunta: ¿Qué ventaja tiene usar `{{baseUrl}}` en lugar de la URL completa?

Adaptación al snapshot final M3

Define `baseUrl=http://localhost:8080` en un entorno exportable y usa `{{baseUrl}}` en las peticiones.

Verificación

Comprueba **M3.local.postman_environment.json** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 8 - Crear una petición GET en Postman

Dentro de la carpeta "Alumnos": 1. Método: GET. 2. URL: `{{baseUrl}}/api/v1/alumnos`. 3. Botón Send. Verás la respuesta en el panel inferior: código 200, tiempo, tamaño y cuerpo JSON.

Pregunta: ¿Qué información muestra Postman sobre la respuesta además del cuerpo?

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduces una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 9 - Crear una petición POST en Postman

Dentro de la carpeta "Alumnos": 1. Método: POST. 2. URL: `{{baseUrl}}/api/v1/alumnos`. 3. Pestaña Body > raw > seleccionar JSON. 4. Introducir el cuerpo:

```
{
  "nombre": "Ana",
  "apellidos": "García López",
  "dni": "12345678A",
```

```

    "fechaNacimiento": "2010-05-12",
    "curso": "5º Primaria"
  }

```

5. Botón Send. Verás la respuesta 201 con la cabecera Location y el recurso creado.

Pregunta: ¿Dónde se ven las cabeceras de la respuesta en Postman?

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduces una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 10 - Añadir un test en Postman

Vamos a añadir un test a la petición POST. En la pestaña **Tests** de la petición, escribe:

```

pm.test("Status code is 201", function () {
  pm.response.to.have.status(201);
});

pm.test("Response has id", function () {
  const jsonData = pm.response.json();
  pm.expect(jsonData).to.have.property("id");
});

pm.test("Location header exists", function () {
  pm.response.to.have.header("Location");
});

```

Guarda y pulsa **Send**. En la pestaña **Test Results** verás los tres tests en verde.

Pregunta: ¿Qué ventaja tiene escribir tests en Postman en lugar de comprobar la respuesta manualmente?

Adaptación al snapshot final M3

Añade un script que compruebe status y una propiedad JSON. Postman complementa, no sustituye, los tests Java.

Verificación

Comprueba `pm.test(...)` y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 11 - Errores comunes del ejercicio

Error	Causa	Solución
400 Bad Request en un @RequestParam	El parámetro quedó obligatorio	Usar <code>required = false</code> o <code>defaultValue</code>

Error	Causa	Solución
400 Bad Request en un @RequestHeader @CookieValue no captura la cookie	Falta la cabecera obligatoria	Usar required = false cuando corresponda
400 Bad Request con varios cursos	La cookie no se envía o el nombre no coincide	Verificar nombre y envío
Postman no envía el cuerpo	El cliente no repite correctamente el parámetro	Usar ?cursos=5º&cursos=6º
Postman no envía cabeceras	Body no está configurado como JSON	Seleccionar raw > JSON
Falla un test de Postman	No se añadieron en la petición	Configurarlas en Headers
415 Unsupported Media Type en Postman	Código o campo esperado no coinciden	Revisar el script y la respuesta real
	Falta Content-Type	Seleccionar JSON para que Postman añada la cabecera

Adaptación al snapshot final M3

Conserva la intención del paso y aplícala sobre las clases del paquete funcional correspondiente. No dupliques capas ni introduzcas una segunda implementación paralela.

Verificación

Comprueba **comportamiento HTTP o de test coherente con el paso** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Paso 12 - Reto resuelto - Endpoint que devuelve información completa de la petición

Reto: Añadir un endpoint `/api/v1/alumnos/debug` que devuelva toda la información de la petición: método, URL, cabeceras, cookies y query parameters. Solución paso a paso: Paso 1: Añadir el método al controlador:

```
@GetMapping("/debug")
public Map<String, Object> debug(
    HttpServletRequest request,
    @RequestHeader Map<String, String> cabeceras,
    @CookieValue Map<String, String> cookies,
    @RequestParam Map<String, String> parametros) {
    return Map.of(
        "metodo", request.getMethod(),
        "url", request.getRequestURL().toString(),
        "queryString", request.getQueryString() != null
            ? request.getQueryString() : "",
        "cabeceras", cabeceras,
        "cookies", cookies,
        "parametros", parametros
    );
}
```

`HttpServletRequest request` es un objeto que Spring inyecta automáticamente. Contiene toda la información de la petición: método, URL, query string, etc. `@RequestHeader Map<String, String> cabeceras` captura todas las cabeceras. `@CookieValue Map<String, String> cookies` captura todas las cookies. `@RequestParam Map<String, String> parametros` captura todos los query parameters. El método devuelve un `Map` con toda la información. Paso 2: Reiniciar y probar:

```
curl "http://localhost:8080/api/v1/alumnos/debug?curso=5º&dni=12345678A" \
-H "User-Agent: MiCliente/1.0" \
-b "preferencias=es-ES"
```

Verás un JSON con toda la información de la petición:

```
{
  "metodo": "GET",
  "url": "http://localhost:8080/api/v1/alumnos/debug",
  "queryString": "curso=5º&dni=12345678A",
  "cabeceras": {
    "host": "localhost:8080",
    "user-agent": "MiCliente/1.0",
    "...": "..."
  },
  "cookies": {
    "preferencias": "es-ES"
  },
  "parametros": {
    "curso": "5º",
    "dni": "12345678A"
  }
}
```

Pregunta: ¿Qué información de la petición te parece más útil para depurar?

Adaptación al snapshot final M3

Devuelve método, path, query, cabeceras seleccionadas y cookie en un endpoint de diagnóstico controlado.

Verificación

Comprueba **GET /api/v1/alumnos/request-info** y ejecuta `./mvnw test` (o `mvnw.cmd test` en Windows). Si el paso modifica HTTP, repite también la petición manual y compara status, cabeceras y cuerpo con lo explicado arriba.

Resultado esperado de la práctica

Al final del ejercicio, deberías tener:

- Endpoints que usan `@RequestParam`, `@RequestHeader` y `@CookieValue`.
- Un endpoint que acepta listas como parámetro.
- Postman instalado y configurado.
- Una colección en Postman con peticiones para la API.
- Un entorno con la variable `baseUrl`.
- Tests en Postman para verificar las respuestas.

Resumen técnico del ejercicio

El ejercicio ha demostrado:

- `@RequestParam`: query parameters, listas, mapas, valores por defecto.
- `@RequestHeader`: cabeceras, opcionales, mapas.
- `@CookieValue`: cookies, opcionales, mapas.
- `HttpServletRequest`: objeto con toda la información de la petición.
- Postman: instalación, colecciones, entornos, tests.
- Buenas prácticas: organización, variables de entorno, tests.

Pregunta final

¿Qué evidencia mínima demuestra que la práctica 3.6 quedó integrada sin romper M2?

Respuesta razonada

La suite completa debe seguir verde y, además, una petición positiva y otra negativa representativas del punto deben producir el status y el cuerpo previstos. Esa combinación verifica regresión y comportamiento nuevo.